

Тетрадь тестировщика

От нуля до собеседования на стажировку QA в R-Vision.
Теория, разобранная на живом проекте, практика с
заготовками и ответами.

Сквозной проект тетради: тест-набор для «Заявки на пропуск» в systemburo

Целевая вакансия: стажёр-тестировщик, R-Vision (ручное тестирование с уклоном в автоматизацию)

Стек по вакансии: Playwright, TypeScript, Git, SQL, API, DevTools, Allure, Confluence

Составлено 6 сентября 2026 года

Содержание

0	Как устроена тетрадь	сквозной проект, значки, дорожная карта на неделю
1	Тестировщик: роль, команда, R-Vision	зачем нужен, кто есть кто, продукты и стек компании
2	База: термины, уровни, виды	дефект и отказ, пирамида, severity и priority
3	Тест-дизайн	классы, границы, таблица решений, состояния, pairwise
4	Документы тестировщика	чек-лист, тест-кейс, баг-репорт, жизненный цикл дефекта
5	Как устроен веб	DNS, порты 80 и 443, HTTP и HTTPS, cookie и токены, кэш, CORS
6	Как устроен продукт изнутри	слои, микросервисы, очереди, Docker и Kubernetes
7	Команда: процесс, Git, окружения	спринт, критерии приёмки, ветки и пул-реквесты, dev-test-prod
8	Логи и мониторинг	уровни логов, идентификатор запроса, что прикладывать к дефекту
9	Домен кибербезопасности	SOC, событие и инцидент, корреляция, уязвимости, плейбуки
10	Веб под капотом	DevTools, DOM, локаторы, Page Object
11	API: проверка без браузера	методы, коды, конверт ответа, JWT, REST и GraphQL
12	SQL: что реально сохранилось	SELECT, JOIN, GROUP BY, проверки после действия
13	TypeScript по делу	типы, интерфейсы, async и await, классы для POM
14	Playwright	локаторы, автоожидания, фикстуры, моки, трейсы
15	CI, отчёты, флаки	конвейер, разбор падений, Allure и TestOps
16	Сборка pet-проекта	структура репозитория, README, критерии готовности
17	Собеседование	карта требований, типовые вопросы, что спросить самому
A	Ответы и решения	разборы всех заданий П1-П22 и тестов Т1-Т14
Б	Шпаргалки	коды ответов, Playwright, SQL, шаблон баг-репорта

0 Как устроена тетрадь

Тетрадь ведёт от «не знаю, что такое дефект» до готового тест-набора, который не стыдно показать на собеседовании. Теории ровно столько, чтобы понимать, что делаешь и зачем; всё остальное время занимает практика.

Сквозной проект

Все разделы работают на один результат. Мы берём одну функцию реального проекта **systemburo** (бюро пропусков: Go + PostgreSQL на бэкенде, Vue 3 на фронте) — **подачу заявки на пропуск** — и по ходу тетради собираем для неё полный набор артефактов тестировщика:

Раздел	Что делаем	Что появляется в наборе	
3. Тест-дизайн	вт	разбиваем поля формы на классы и границы	чек-лист проверок
4. Документация	вт	оформляем проверки и найденный дефект	6 тест-кейсов, 1 баг-репорт
10. Веб и локаторы	ср	снимаем селекторы со страницы	карта локаторов
11. API	ср	проверяем запрос создания заявки	API-проверка
12. SQL	чт	убеждаемся, что данные легли в базу	SQL-запрос проверки
8-9. TS и Playwright	пишем Page Object и автотесты	3 автотеста + POM	
16. Сборка	сб	складываем всё в репозиторий с отчётом	готовый pet-проект

Именно такой pet-проект закрывает сразу четыре строки требований вакансии: «опыт разработки учебных или pet-проектов», «знание Page Object и локаторов», «Git», «плюсом TypeScript и Playwright».

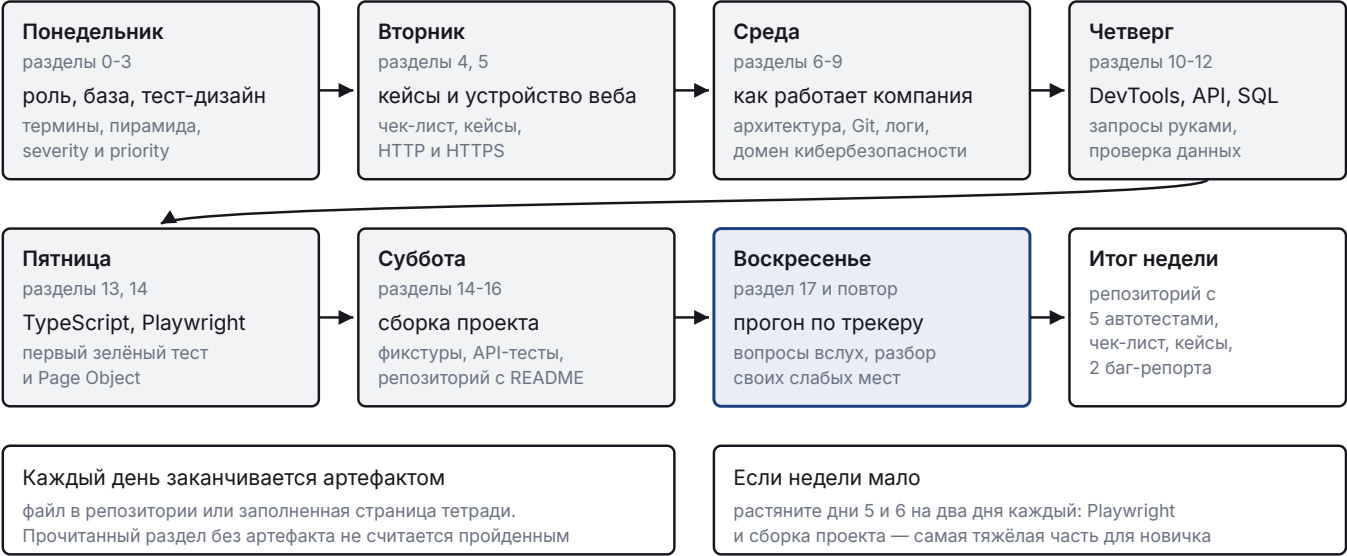
Значки на полях

Термин	определение, которое спросят на собеседовании
Пример	тот же термин на конкретной ситуации
В проекте systemburo	как это выглядит в живом коде, а не в учебнике
Практика ПН	задание с заготовкой: дописать, дополнить, исправить
Тест TN	вопрос с вариантами ответа для самопроверки
Ловушка	ошибка новичка, которую видно на собеседовании сразу

Ответы на все задания и тесты — в приложении А. Подглядывать сразу бессмысленно: ценность задания в попытке, а не в правильной строчке.

Дорожная карта: одна неделя, с понедельника по воскресенье

План рассчитан на семь дней подряд: по 4-5 часов в будни и по 6-7 в выходные. Это интенсив, а не фон: каждый день опирается на предыдущий, и пропуск дня рвёт цепочку. Если день выпал, сдвигайте всю неделю, а не догоняйте два дня за один.



Неделя: семь дней, каждый со своим результатом

День	Разделы	Часы	Что должно появиться к вечеру
Понедельник	0, 1, 2, 3	5	трекер, ответы на Т1-Т3, задания П1-П4
Вторник	4, 5	5	чек-лист, 6 тест-кейсов, баг-репорт, разбор URL и HTTPS
Среда	6, 7, 8, 9	5	критерии приёмки, git-последовательность, разбор лога, словарь ИБ
Четверг	10, 11, 12	5	карта локаторов, каркас POM, запросы через curl, SQL-проверки
Пятница	13, 14	5	установленный Playwright, первый зелёный тест, LoginPage
Суббота	14, 15, 16	6-7	фикстура, три UI-теста, два API-теста, репозиторий с README
Воскресенье	17, повторение	6	ответы вслух на вопросы 17.2, закрытый трекер, отклик на вакансию

Трекер прогресса

Отмечайте по ходу. Третья колонка — самая важная: задание считается сделанным, только когда вы сверились с ответом в приложении А и поняли расхождение, если оно было.

Раздел	День	Задания	Прочитан	Практика	Сверено
1. Тестировщик: роль и команда	пн	T1	☐	☐	☐
2. База: термины, уровни, виды	пн	П1, T2	☐	☐	☐
3. Тест-дизайн	пн	П2-П4, T3	☐	☐	☐
4. Документы тестировщика	вт	П5, П6, T4	☐	☐	☐
5. Как устроен веб	вт	П7, П8, T5	☐	☐	☐
6. Как устроен продукт изнутри	ср	П9, T6	☐	☐	☐
7. Команда: процесс, Git, окружения	ср	П10, П11, T7	☐	☐	☐
8. Логи и мониторинг	ср	П12, T8	☐	☐	☐
9. Домен кибербезопасности	ср	T9	☐	☐	☐
10. Веб под капотом	чт	П13, П14, T10	☐	☐	☐
11. API	чт	П15, П16, T11	☐	☐	☐
12. SQL	чт	П17, T12	☐	☐	☐
13. TypeScript по делу	пт	П18, П19	☐	☐	☐
14. Playwright	пт	П20, П21, T13	☐	☐	☐
15. CI, отчёты, флаки	сб	T14	☐	☐	☐
16. Сборка pet-проекта	сб	П22	☐	☐	☐
17. Собеседование	вс	—	☐	☐	☐

Что понадобится установить

- **Node.js 20+** и **npm** — среда для Playwright.
- **VS Code** — редактор, расширение Playwright Test for VSCode.
- **Git** и аккаунт на **GitHub** — репозиторий pet-проекта.
- **Браузер Chrome или Firefox** — DevTools открывается по F12.
- По желанию: **Postman** для API и **DBeaver** для SQL. Оба заменяются командной строкой.

1 Тестировщик: роль, команда, R-Vision

Начинать с техники бессмысленно, пока непонятно, зачем в команде отдельный человек и за что ему платят.

1.1 Зачем нужен тестировщик

Разработчик пишет код и проверяет, что *его сценарий* работает. Тестировщик проверяет, что *система* работает: в том числе там, где данные пришли неожиданные, пользователь нажал не туда, а две функции пересеклись боком.

Тестирование — проверка соответствия того, что система делает, тому, что она должна делать, с целью найти расхождение раньше пользователя.

Важная поправка: тестирование не доказывает, что ошибок нет. Оно показывает, что найденные ошибки есть, а ненайденные могут существовать. Полный перебор невозможен: у формы из пяти полей комбинаций больше, чем жизни на их проверку. Отсюда всё ремесло тест-дизайна — как проверить малым числом сценариев максимум риска.

В ПРОЕКТЕ SYSTEMBURO

В systemburo есть 7652 автотеста, зелёный CI и всё равно случались провалы на проде: деплой падал на сборке nginx, интерфейс не разворачивал ответ API формы `{table:{}}`. Вывод, записанный в правилах проекта: «зелёный CI не равно работает». Автотесты проверяют то, что в них заложили, а не то, что важно пользователю.

1.2 Кто есть кто в тестировании

Роль	Чем занимается
Ручной тестировщик (QA manual)	проверяет функции руками, пишет кейсы и баг-репорты, исследует новое
Автоматизатор (AQA)	переносит повторяющиеся проверки в код, поддерживает фреймворк тестов
Инженер по нагрузке	проверяет, держит ли система поток данных и не деградирует ли между релизами
AppSec-инженер	ищет уязвимости в своём же продукте: пентест, фаззинг, разбор сканеров
Лид автоматизации	фреймворк, стратегия, люди

В вакансии стажёра R-Vision написано «ручное тестирование с уклоном в автоматизацию». Это означает: начинаете с кейсов и баг-репортов, но растёте в сторону кода. Отдельной вакансии ручного тестировщика у компании нет вообще — единственный вход в направление лежит через автоматизацию.

1.3 Куда именно вы идёте: R-Vision

Компания с 2011 года делает системы кибербезопасности, около 442 сотрудников, выручка за 2025 год 2,72 млрд рублей. Больше половины штата — департамент разработки. Продукты объединены в экосистему R-Vision EVO:

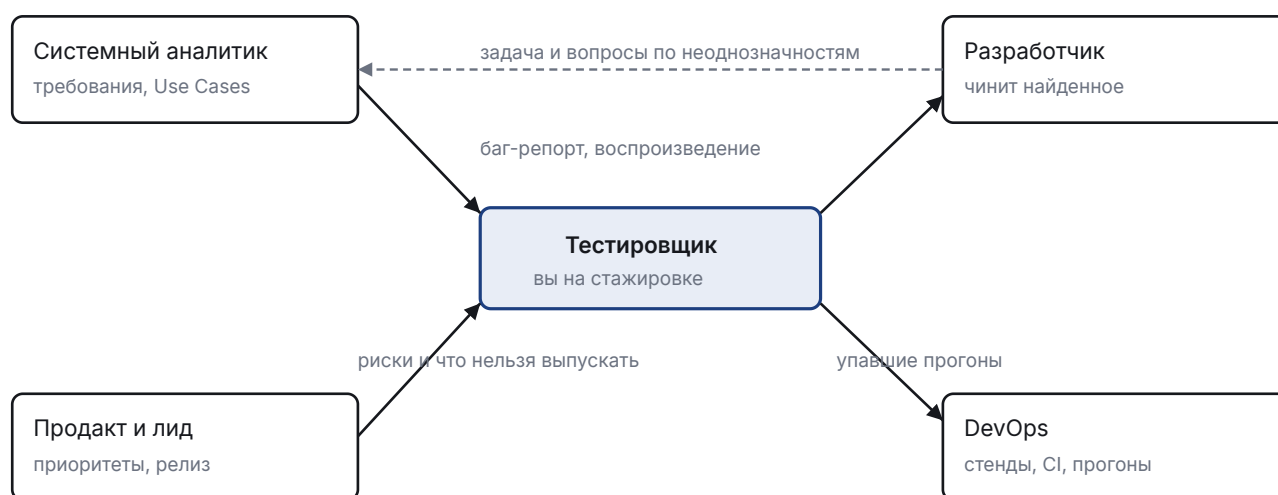
Продукт	Что делает	Что это значит для тестировщика
SIEM	собирает и коррелирует события безопасности	потoki в 10 000 событий в секунду, хранение в ClickHouse
SOAR	автоматизирует реагирование на инциденты, плейбуки	сценарии с ветвлением, интеграции с чужими системами
SGRC	управление рисками и соответствием стандартам	много форм, ролей и отчётов
VM	управление уязвимостями	сканеры, большие таблицы, фильтры
TIP	данные о киберугрозах	миллион сущностей на источник, часть на Rust
TDP	ловушки для атакующих	сетевые сценарии
CMDB, ITSM, ITAM	ИТ-учёт и процессы	связи объектов, справочники

Технически это веб-интерфейсы на **React** и **TypeScript** с микрофронтендной архитектурой поверх бэкенда на **Node.js (NestJS)**, местами на **Rust**, с **PostgreSQL** и **ClickHouse**, очередями **NATS**, доставкой на **Kubernetes** и сборкой в **GitLab CI**. Автотесты — на **Playwright** с **TypeScript**, отчёты в **Allure TestOps**.

ЛОВУШКА

Не путайте два стека в их вакансиях. На карьерном сайте у инженера по автоматизации упомянуты Java, Kotlin, Selenide, а в живой вакансии QA Automation и у лида — Playwright с TypeScript. Это похоже на незаконченный переезд с одного фреймворка на другой. Учить надо то, что в живой вакансии: Playwright.

1.4 С кем работает тестировщик



Связи тестировщика в продуктовой команде

В вакансии стажёра прямо записано «взаимодействие с аналитиками и разработчиками» и «анализ документации — выявление пропусков и неоднозначностей». Это значит: от вас ждут вопросов к постановке до того, как код написан. Найденная в требованиях дыра стоит компании в десятки раз дешевле, чем тот же дефект, найденный на проде.

1.5 Как выглядит рабочая неделя

- Стендап каждый день, 10-15 минут: что делал, что буду, что мешает.
- Планирование раз в спринт (обычно 2 недели): команда разбирает задачи.
- Груминг или разбор бэклога: аналитик рассказывает задачу, вы задаёте вопросы «а что если».
- Ретро в конце спринта: что улучшить в процессе.
- Между этим — работа: кейсы, прогоны, баг-репорты, автотесты, разбор упавшего в CI.

T1 Проверьте себя

1. Тестирование доказывает отсутствие ошибок в программе. Верно или нет?
2. Какая роль пишет автотесты и поддерживает фреймворк: QA manual, AQA, DevOps?
3. Что означает «анализ документации» в обязанностях стажёра R-Vision?
 - а) читать инструкции пользователя
 - б) искать в требованиях пропуски и противоречия до начала разработки
 - в) писать документацию вместо аналитика

ИТОГ РАЗДЕЛА

Тестировщик отвечает не за «нажать все кнопки», а за информацию о качестве: что сломано, чем это грозит и можно ли выпускать. В R-Vision стажёр входит через ручное тестирование, но растёт в автоматизацию на Playwright и TypeScript, потому что другого пути в компании нет.

МОИ ЗАМЕТКИ

Главное из раздела своими словами:

Что спросить у ментора или на собеседовании:

2 База: термины, уровни, виды

Словарь, на котором говорит команда. Половина вопросов на собеседовании — отсюда.

2.1 Ошибка, дефект, отказ

Ошибка (error) — действие человека, приведшее к неверному результату. Программист перепутал знак сравнения.

Дефект (defect, bug) — сам изъян в коде или документе. Условие $\geq$ вместо $>$.

Отказ (failure) — наблюдаемое неверное поведение системы. Заявка на 30 дней проходит, хотя лимит 30 дней должен её отклонять.

Цепочка всегда одна: человек ошибся → в продукте появился дефект → при выполнении он проявился как отказ. Дефект может годами лежать в коде и не давать отказа, пока не выполнится нужное условие.

Верификация — «делаем ли мы продукт правильно» (соответствие требованиям).

Валидация — «делаем ли мы правильный продукт» (решает ли он задачу пользователя).

Форма подачи заявки сделана ровно по макету и требованиям — верификация пройдена. Но охраннику на КПП нужна фамилия водителя, которой в форме нет вообще, и заявку приходится уточнять по телефону — валидация провалена.

2.2 Уровни тестирования и пирамида



Пирамида: чем выше уровень, тем меньше тестов и тем дороже каждый

В ПРОЕКТЕ SYSTEMBURO

Счётчики systemburo на сегодня: 2204 теста бэкенда в 396 файлах (Go), 5258 тестов фронтенда в 587 файлах (Vitest), плюс несколько десятков E2E-сценариев на Playwright. Форма пирамиды соблюдена: основание из быстрых юнитов, узкая верхушка из медленных браузерных прогонов.

ЛОВУШКА

Перевернутая пирамида («мороженое») — когда E2E-тестов больше, чем юнитов. Прогон занимает часы, половина падений — не баги, а нестабильность, и команда перестаёт верить красному CI. Если на собеседовании спросят, почему нельзя всё покрыть E2E, ответ именно такой.

2.3 Виды тестирования

Вид	Когда применяется	Пример на заявке
Смоук	сразу после сборки: живо ли вообще	вход в систему, открытие формы, отправка одной заявки
Регресс	после изменений: не сломалось ли старое	прогон всех кейсов подачи после правки статусов
Ретест	после починки конкретного бага	повторяем шаги из баг-репорта
Санити	узкая проверка одной области	только фильтр по датам, который вчера правили
Исследовательское	когда нет требований или мало времени	час свободного «а что если» по новой форме
Статическое	до запуска кода	чтение требований и поиск противоречий

Разделение по знанию устройства: **чёрный ящик** — проверяем только вход и выход, кода не видим; **белый ящик** — знаем реализацию и целимся в ветки кода; **серый ящик** — знаем структуру данных и API, но тестируем снаружи. Тестировщик на продукте чаще всего работает в сером: интерфейс снаружи, но с пониманием, какой запрос уходит и что лежит в базе.

2.4 Severity и Priority

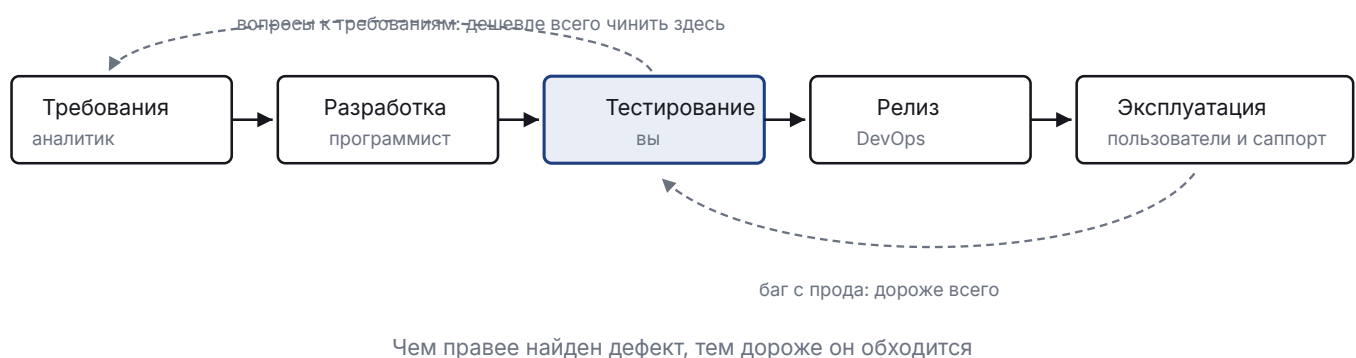
Severity (серьёзность) — насколько сильно дефект ломает систему. Объективная характеристика, её называет тестировщик.

Priority (приоритет) — насколько срочно чинить. Решение бизнеса, его ставит продакт или лид.

Ситуация	Severity	Priority	Почему
Опечатка в названии компании на главной	низкая	высокий	видят все клиенты, стыдно
Падение экспорта отчёта, которым пользуются раз в год	высокая	низкий	ломает функцию, но ждать может
Не отправляется заявка ни у кого	блокер	высокий	продукт не выполняет свою работу

Классическая шкала severity: **блокер** (работать нельзя), **критический** (ключевая функция сломана), **значительный** (функция работает не так), **незначительный** (неудобно, но обходится), **тривиальный** (косметика).

2.5 Жизненный цикл разработки и место проверок



П1 Разложите по полочкам

Дан кусок требования к форме подачи заявки:

Поле «Дата въезда» обязательно.
Заявку можно подать не более чем на 30 дней.
Прошедшую дату выбрать нельзя.

Заполните таблицу, дописав пустые ячейки:

Что проверяем	Вид тестирования	Severity, если сломано
Форма вообще открывается и заявка отправляется	смоук	блокер
Дата на 31 день вперёд отклоняется	_____	_____
После правки календаря старые сценарии не сломались	_____	_____
В требовании не сказано, считать ли 30 дней от сегодня или от даты подачи	_____	_____

Т2 Проверьте себя

1. Программист перепутал знак сравнения — это ошибка, дефект или отказ?
2. Кто ставит Priority: тестировщик, продакт, разработчик?
3. Какой вид тестирования запускают первым после новой сборки?
а) регресс б) смоук в) исследовательское
4. Почему нельзя покрыть весь продукт только E2E-тестами? Ответьте одним предложением.

ИТОГ РАЗДЕЛА

Ошибка порождает дефект, дефект проявляется отказом. Verification про соответствие требованиям, validation про пользу. Пирамида: много дешёвых юнитов, мало дорогих E2E. Severity объективна и ваша, Priority решает бизнес.

МОИ ЗАМЕТКИ

Термины, которые пока путаю:

Свой пример на severity и priority:

3 Тест-дизайн: как проверить много малым числом сценариев

Главный профессиональный навык. Отличает тестировщика от человека, который просто тыкает кнопки.

3.1 Почему нельзя проверить всё

Форма подачи заявки в systemburo: организация, сотрудник, дата въезда, дата выезда, цель визита, машина. Даже если у каждого поля всего 10 осмысленных значений, комбинаций миллион. Проверять их все не хватит жизни. Тест-дизайн — набор техник, которые отбирают из миллиона десятков проверок, покрывающих те же риски.

3.2 Классы эквивалентности

Класс эквивалентности — группа входных значений, которые система обрабатывает одинаково. Достаточно проверить одно значение из класса: остальные поведут себя так же.

Поле «Срок действия пропуска», допустимо от 1 до 30 дней.

Классы: **меньше 1** (0, -5) — невалидный; **от 1 до 30** — валидный; **больше 30** (31, 100) — невалидный; **не число** («абв», пустое) — невалидный.

Вместо тридцати проверок берём четыре: одно значение из каждого класса.

3.3 Анализ граничных значений

Ошибки живут на границах: перепутанный $<$ и $\leq$ — самый частый дефект в проверках. Поэтому к классам добавляют значения по краям: **граница - 1**, **граница**, **граница + 1**.



ЛОВУШКА

Типичная ошибка новичка — проверить «нормальное» значение 15 и успокоиться. Дефект «на 30 дней заявка не проходит, хотя должна» живёт ровно на границе, и середина диапазона его не поймает.

3.4 Таблица решений

Когда результат зависит от нескольких условий сразу, помогает таблица: столбцы — комбинации, строки — условия и результат.

В ПРОЕКТЕ SYSTEMBURO

Видимость заявки в systemburo зависит от трёх вещей: пользователь — суперадмин или нет; заявка принадлежит его организации или чужой; заявка активна или в архиве. Это ровно тот случай, когда без таблицы решений легко пропустить комбинацию.

Условие	K1	K2	K3	K4	K5
Суперадмин	да	нет	нет	нет	нет
Своя организация	—	да	да	нет	нет
Заявка в архиве	—	нет	да	нет	да
Ожидаемый результат	видит	видит	видит в архиве	404	404

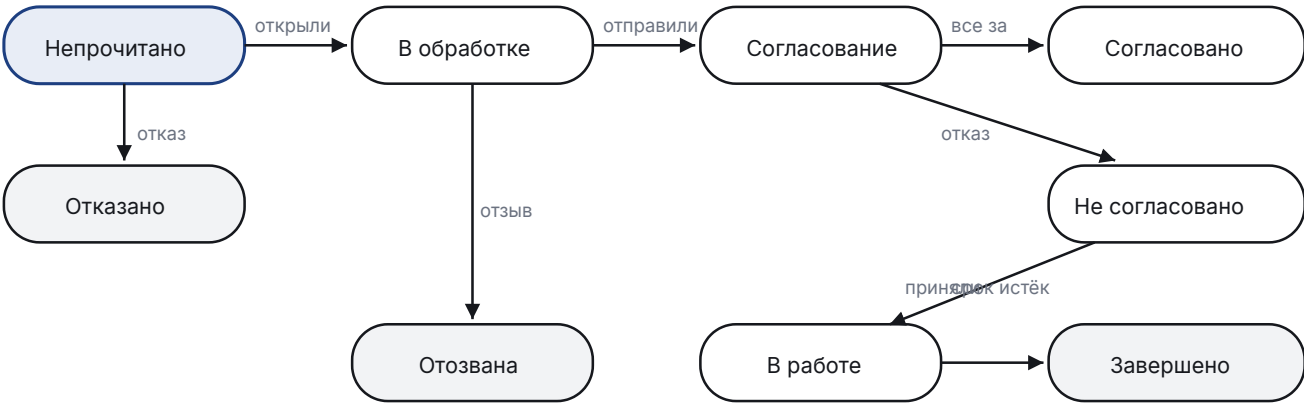
Прочерк означает «значение не важно»: суперадмин видит всё в любом случае, и раздувать таблицу его комбинациями смысла нет.

3.5 Диаграмма состояний и переходов

Техника для объектов, которые живут во времени и меняют статус. Проверяем не только «каждый статус достижим», но и «запрещённые переходы действительно запрещены».

В ПРОЕКТЕ SYSTEMBURO

Статусы заявки в systemburo заданы в коде константами: Непрочитано, В обработке, Согласование, Согласовано, Не согласовано, В работе, Завершено, Отказано, Отозвана. Отдельно отмечено, что у статуса «Отозвана» обратного пути нет.



Серым — терминальные состояния: из них переходов нет. Проверять надо и их: из «Отозвана» заявка не должна возвращаться в работу ни через интерфейс, ни через прямой запрос к API.

Жизненный цикл заявки: узлы — статусы, стрелки — разрешённые переходы

3.6 Попарное тестирование (pairwise)

Когда параметров много, полный перебор не нужен: большинство дефектов проявляется от взаимодействия двух факторов, а не пяти сразу. Pairwise подбирает набор комбинаций так, чтобы каждая пара значений встретилась хотя бы раз.

Три параметра по три значения: браузер (Chrome, Firefox, Safari), роль (админ, сотрудник, охранник), тема (светлая, тёмная, системная). Полный перебор — 27 прогонов. Pairwise даёт 9, покрывая все пары. Инструменты: PICT, allpairs, онлайн-генераторы.

3.7 Какую технику когда брать

Ситуация	Техника
Поле ввода с диапазоном или форматом	классы эквивалентности + границы
Результат зависит от нескольких флагов и ролей	таблица решений
Объект меняет статусы	диаграмма состояний
Много независимых настроек	pairwise
Требований нет, время ограничено	исследовательское + предугадывание ошибок

П2 Классы и границы своими руками

Поле «Номер автомобиля» в заявке: русский формат A123BV777, буквы только из разрешённых (АВЕК-МНОРСТУХ), регион 2 или 3 цифры. Заполните таблицу:

Класс	Пример значения	Ожидаемый результат
Валидный, регион 3 цифры	A123BV777	принимается
Валидный, регион 2 цифры	_____	_____
Запрещённая буква	Ж123BV77	_____
_____	A12BV77	отклоняется
Пустое поле	_____	_____
Латиница вместо кириллицы	A123BV77	_____

Последняя строка — ловушка. Подумайте, каким должно быть поведение и почему это вопрос к аналитику, а не к вам.

П3 Достройте таблицу решений

Кнопка «Отозвать заявку» показывается, если: пользователь — автор заявки, заявка не в архиве, статус не терминальный. Достройте колонки К3 и К4 и впишите результат:

Условие	К1	К2	К3	К4
Пользователь — автор	да	нет	да	да
Заявка не в архиве	да	да	нет	да
Статус не терминальный	да	да	да	нет
Кнопка видна	да	нет	_____	_____

П4 Переходы, которых быть не должно

По схеме жизненного цикла выпишите три перехода, которые система обязана запрещать, и придумайте, как проверить каждый не через интерфейс, а напрямую запросом к API. Заготовка:

1. Из «Отозвана» → «В работе». Проверка: PATCH /applications/{id}/status с телом {"status": "В работе"} должен вернуть _____ (какой код?)
2. Из «Завершено» → _____. Проверка: _____
3. Из «Непрочитано» → _____. Проверка: _____

Т3 Проверьте себя

1. Диапазон допустимых значений 10-99. Назовите шесть значений по границам.
2. Какую технику выбрать для проверки прав доступа, зависящих от роли, владельца и статуса?
3. Pairwise экономит прогоны за счёт допущения, что:
 - а) большинство дефектов вызывается взаимодействием двух параметров
 - б) параметры не влияют друг на друга
 - в) можно тестировать только популярные комбинации
4. Что важнее проверить в диаграмме состояний: что разрешённые переходы работают или что запрещённые не работают?

ИТОГ РАЗДЕЛА

Классы и границы — для полей. Таблица решений — для комбинаций условий. Диаграмма состояний — для статусов, причём запрещённые переходы важнее разрешённых. Pairwise — когда параметров много. Из этих техник и рождается чек-лист сквозного проекта.

МОИ ЗАМЕТКИ

Какую технику мне труднее всего применять и почему:

Поля из знакомого мне интерфейса, разложенные на классы:

4 Документы тестировщика: чек-лист, кейс, баг-репорт

Проверка, о которой никто не узнал, не считается. Половина работы — внятно записать.

4.1 Чек-лист и тест-кейс

Чек-лист — список того, что надо проверить, без подробностей. Быстро пишется, требует знания системы.

Тест-кейс — пошаговый сценарий с конкретными данными и ожидаемым результатом. Дольше пишется, но его выполнит любой новичок и получит тот же ответ.

Берём	Когда
Чек-лист	своя команда, времени мало, функция знакомая, идёт исследовательский прогон
Тест-кейс	регресс, передача другому человеку, приёмка, сертификация, основа для автотеста

Структура тест-кейса

ID:	APP-07
Название:	Отклонение заявки со сроком больше 30 дней
Приоритет:	высокий
Предусловия:	пользователь sergey вошёл в систему, у организации «Ромашка» есть активное вложение
Тестовые данные:	дата въезда = сегодня, дата выезда = сегодня + 31 день
Шаги:	1. Открыть «Подать заявку» 2. Выбрать организацию «Ромашка» 3. Заполнить даты из тестовых данных 4. Нажать «Отправить»
Ожидаемый результат:	заявка не создана, под полем «Дата выезда» показано сообщение «Срок пропуска не может превышать 30 дней», запрос POST /applications не отправлен
Постусловия:	данные не изменились, форма осталась заполненной

ЛОВУШКА

Ожидаемый результат «должно работать корректно» — не результат. Проверяемая формулировка называет конкретный признак: текст сообщения, код ответа, число строк, статус в базе. Если по вашему кейсу двое исполнителей могут прийти к разным выводам, кейс написан плохо.

4.2 Баг-репорт

Цель баг-репорта — чтобы разработчик воспроизвёл дефект без вас и починил, не задавая вопросов. Всё остальное вторично.

Поле	Что писать
Заголовок	что сломано, где и при каких условиях, одной строкой
Окружение	стенд, версия сборки, браузер, роль пользователя
Предусловия	что должно быть в системе до шагов
Шаги	пронумерованные, минимальные, с конкретными данными
Фактический результат	что происходит, с текстом ошибки и кодом ответа
Ожидаемый результат	что должно происходить и на основании чего (ссылка на требование)
Вложения	скриншот, видео, лог консоли, трейс Playwright, запрос из Network
Severity	ваша оценка серьёзности

Плохо и хорошо

ПЛОХО

Заголовок: Заявка не работает

Шаги: подал заявку, ничего не произошло

Факт: не работает

ХОРОШО

Заголовок: Заявка со сроком 30 дней отклоняется, хотя граница включительная

Окружение: staging, сборка 2026.09.06, Chrome 140, роль «Сотрудник»

Предусловия: пользователь sergey, организация «Ромашка», активное вложение

Шаги:

1. Открыть «Подать заявку»
2. Дата въезда = 06.09.2026, дата выезда = 05.10.2026 (ровно 30 дней)
3. Нажать «Отправить»

Факт: под полем «Дата выезда» сообщение «Срок пропуска не может превышать 30 дней», POST /api/applications не отправляется

Ожидаемо: заявка создаётся, 30 дней входят в допустимый диапазон (требование ТЗ 4.2.1: «не более 30 дней»)

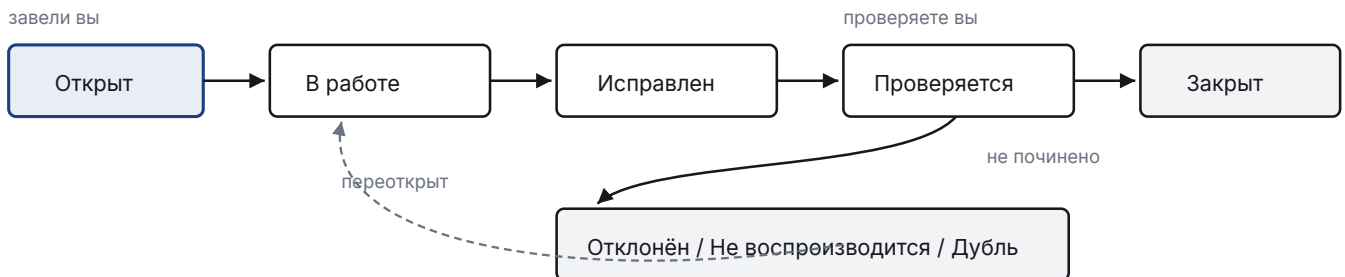
Вложения: скриншот, HAR-запись вкладки Network

Severity: значительный

В ПРОЕКТЕ SYSTEMBURO

Из уроков systemburo: часть найденных дефектов не ловилась ни одним из 7652 автотестов, потому что проявлялась только на реальных данных и на конкретной платформе. Поэтому в баг-репорте важны окружение и данные: без них дефект «не воспроизводится» и закрывается как несуществующий.

4.3 Жизненный цикл дефекта



Путь дефекта в трекере. Закрывает баг тестировщик, а не разработчик

Статус «Не воспроизводится» почти всегда означает, что в репорте не хватило данных: роли, окружения, конкретных значений. Это не спор с разработчиком, а сигнал дописать шаги.

4.4 Где всё это живёт

- **Jira** — задачи и дефекты. Стажёру достаточно уметь завести баг, приложить файл, поменять статус, найти свои задачи фильтром.
- **Confluence** — база знаний: тест-планы, чек-листы, описания функций. В вакансии R-Vision «написание и поддержка статей в Confluence» стоит прямо в задачах.
- **TestOps (Allure)** — хранилище тест-кейсов и результатов прогонов, связывает автотесты с кейсами.

П5 Почините баг-репорт

Дан репорт от стажёра. Перепишите его так, чтобы разработчик воспроизвёл дефект без вопросов. Используйте структуру из 4.2.

Заголовок: Ошибка в архиве
Шаги: зашёл в архив, нажал на заявку, вылезла ошибка
Факт: ошибка 500
Severity: блокер

Подсказки: чего не хватает в заголовке; какие три поля отсутствуют полностью; корректна ли severity, если архив открывают раз в месяц и другие разделы работают.

П6 Первый артефакт сквозного проекта

Соберите чек-лист для формы подачи заявки: не менее 12 пунктов, сгруппированных по блокам. Начало дано, продолжите:

ФОРМА ПОДАЧИ ЗАЯВКИ — ЧЕК-ЛИСТ

Открытие и доступ

- ☐ форма открывается по кнопке «Подать заявку»
- ☐ неавторизованного пользователя редиректит на вход
- ☐ _____

Поля и валидация

- ☐ пустая форма не отправляется, ошибки под каждым полем
- ☐ срок 30 дней принимается (граница включительная)
- ☐ срок 31 день отклоняется
- ☐ _____
- ☐ _____

Отправка

- ☐ после отправки заявка появляется в списке со статусом «Непрочитано»
- ☐ _____

Негатив и края

- ☐ двойной клик по «Отправить» не создаёт две заявки
- ☐ _____
- ☐ _____

Этот файл — первый артефакт вашего рет-проекта. Сохраните его как `checklists/application-form.md` и сделайте первый коммит.

Т4 Проверьте себя

1. Чем чек-лист отличается от тест-кейса одним предложением?
2. Кто переводит дефект в статус «Закрыт»?
3. Что из перечисленного нельзя писать в ожидаемом результате?
 - а) «показано сообщение "Срок пропуска не может превышать 30 дней"»
 - б) «система работает корректно»
 - в) «ответ 201 и заявка видна в списке»
4. Дефект воспроизводится только у роли «Охранник». Куда это писать в репорте?

ИТОГ РАЗДЕЛА

Чек-лист для скорости, кейс для повторяемости. Баг-репорт живёт или умирает по одному критерию: воспроизводится ли он без автора. Ожидаемый результат всегда проверяемый, severity ваша, приоритет не ваш.

МОИ ЗАМЕТКИ

Что я исправил бы в своём первом баг-репорте:

Пункты чек-листа, до которых додумался сам:

5 Как устроен веб

Половина дефектов на веб-продукте объясняется тем, что происходит между нажатием кнопки и появлением ответа. Разберём эту дорогу целиком.

5.1 Клиент и сервер

Клиент — программа, которая просит: браузер, мобильное приложение, скрипт с curl. **Сервер** — программа, которая отвечает. Между ними **протокол** — договор о форме запроса и ответа. Для веба это HTTP.

Ключевое свойство HTTP: он **без состояния**. Сервер не помнит предыдущий запрос. Чтобы система узнавала пользователя, к каждому запросу приходится прикладывать пропуск: cookie с идентификатором сессии или токен в заголовке.

5.2 Что происходит, когда вы вводите адрес

Тестировщику важны шаги 3-5: порт, шифрование и код ответа. Шаг 6 — то, что видно глазами



Путь от адресной строки до нарисованной страницы

5.3 Из чего состоит адрес

<https://staging.rvision.ru:8443/applications?status=new&page=2#top>

схема домен порт путь параметры якорь

Схема задаёт порт по умолчанию: http → 80, https → 443. Явный порт (:8443) нужен только на нестандартных стендах. Параметры после ? — то, что чаще всего ломается: фильтры, сортировка, страницы. Якорь после # на сервер не отправляется вообще: он обрабатывается только в браузере.

Разбор URL по частям

5.4 Порты 80 и 443, HTTP и HTTPS

Порт — номер «двери» на сервере: по одному IP-адресу работает много служб, и порт говорит, какой именно адресован запрос. **80** — стандартный порт HTTP, **443** — HTTPS. Браузер подставляет их сам, поэтому в адресе их не видно.

HTTPS — это тот же HTTP, завернутый в TLS. TLS даёт три вещи: содержимое нельзя прочитать по дороге, нельзя незаметно подменить, и клиент убеждается, что говорит с настоящим сервером — по сертификату, выданному удостоверяющим центром.

Что проверять	Как и зачем
Редирект с HTTP на HTTPS	открыть <code>http://сайт</code> — должен вернуться 301 и адрес смениться на <code>https</code> . Иначе пароль уйдёт открытым текстом
Сертификат	замок в адресной строке, срок действия, совпадение домена. Просроченный сертификат — блокирует: браузер покажет предупреждение вместо продукта
Смешанный контент	страница по HTTPS тянет что-то по HTTP — браузер блокирует, в консоли ошибка <code>mixed content</code> , часть интерфейса не работает
Нестандартные порты	на тестовых стендах <code>:8080</code> , <code>:8081</code> . Проверить, что ссылки внутри продукта не теряют порт

В ПРОЕКТЕ SYSTEMBURO

В `systemburo` фронтенд поднимается на порту 8081, бэкенд — на 8080, а на боевом сервере оба закрыты `nginx`, который слушает 443 и сам перенаправляет с 80. Отсюда типичный дефект стенда: ссылка, сформированная бэкендом, содержит внутренний порт и снаружи не открывается.

5.5 Заголовки: служебная часть запроса

Запрос:	Ответ:
POST /api/applications HTTP/1.1	HTTP/1.1 201 Created
Host: staging.rvision.ru	Content-Type: application/json
Content-Type: application/json	Set-Cookie: session=abc; HttpOnly
Authorization: Bearer eyJhbGci...	Cache-Control: no-store
Accept-Language: ru-RU	Location: /applications/1042

Content-Type	в каком формате тело: JSON, форма, файл
Authorization	пропуск: токен пользователя
Set-Cookie	сервер просит браузер запомнить значение
Cache-Control	можно ли кэшировать ответ и как долго

5.6 Cookie, сессия и токен

Механизм	Как работает	Что проверяет тестировщик
Cookie	браузер хранит пару имя-значение и шлёт её на каждый запрос к домену	флаги <code>HttpOnly</code> и <code>Secure</code> , срок жизни, удаление при выходе
Сессия	в cookie лежит только идентификатор, состояние хранит сервер	выход убивает сессию, чужой идентификатор не работает
Токен (JWT)	подписанная строка, клиент шлёт её в заголовке <code>Authorization</code>	протухание, обновление, доступ с чужим токеном

ЛОВУШКА

Ошибка новичка: проверить выход из системы только по тому, что интерфейс вернулся на форму входа. Настоящая проверка — повторить запрос со старым токеном или cookie: он должен получить 401.

5.7 Кэш: почему «у меня не воспроизводится»

Браузер и промежуточные серверы сохраняют ответы, чтобы не спрашивать заново. Из-за этого исправленный файл может не доехать до пользователя, а тестировщик — увидеть старую версию интерфейса. Что делать: жёсткая перезагрузка (`Ctrl+Shift+R`), галка `Disable cache` в `DevTools`, проверка в приватном окне. Если после сброса кэша дефект исчез — это отдельный дефект про кэширование, а не «показалось».

5.8 Редиректы, CORS и другие частые находки

- 301 и 302 — постоянное и временное перенаправление. Проверяем цепочку: не должно быть петель и лишних шагов, адрес должен сохранять параметры.
- CORS — правило браузера: страница с одного домена не может свободно ходить запросами на другой. В консоли выглядит как «blocked by CORS policy». Для тестировщика это сигнал: фронт и бэк

разведены по разным адресам, а разрешение не настроено.

- 429 — слишком много запросов, сработала защита от перебора. Частая причина странных падений автотестов, гоняющих логин в несколько потоков.

П7 Разберите адрес и ответьте

`https://buro.example.ru/applications?status=new&page=2`

1. Какой порт будет использован и почему его не видно? _____
2. Что произойдёт, если открыть тот же адрес через `http://` ?
Какой код ответа вы ожидаете увидеть в Network? _____
3. Тестировщик открыл страницу, увидел старый интерфейс.
Три действия, чтобы отличить кэш от дефекта: _____
4. В консоли ошибка «mixed content». Что это значит и чей это дефект: фронта, бэка или инфраструктуры? _____

П8 Диагностика по симптому

Для каждой ситуации напишите, куда смотреть и какой дефект предполагаете:

Симптом	Где смотреть и что это может быть
После выхода из системы старая вкладка продолжает показывать данные	
Часть иконок не загрузилась, в консоли ошибки блокировки	
Автотесты падают на входе только при параллельном запуске	
Ссылка из письма открывается, а из приложения — нет	

Т5 Проверьте себя

1. Чем HTTPS отличается от HTTP и какой порт использует каждый?
2. Что означает «HTTP не хранит состояние» и как система тогда узнаёт пользователя?
3. Какая часть URL не отправляется на сервер?
а) путь б) параметры после ? в) якорь после #
4. Сертификат просрочен на день. Severity?

ИТОГ РАЗДЕЛА

HTTP — договор между клиентом и сервером, без памяти о прошлых запросах. HTTPS — он же плюс шифрование TLS, порт 443 против 80. Пропуск пользователя живёт в cookie или токене, и проверять выход надо запросом, а не картинкой. Кэш, CORS и редиректы дают дефекты, которые видно только в DevTools.

МОИ ЗАМЕТКИ

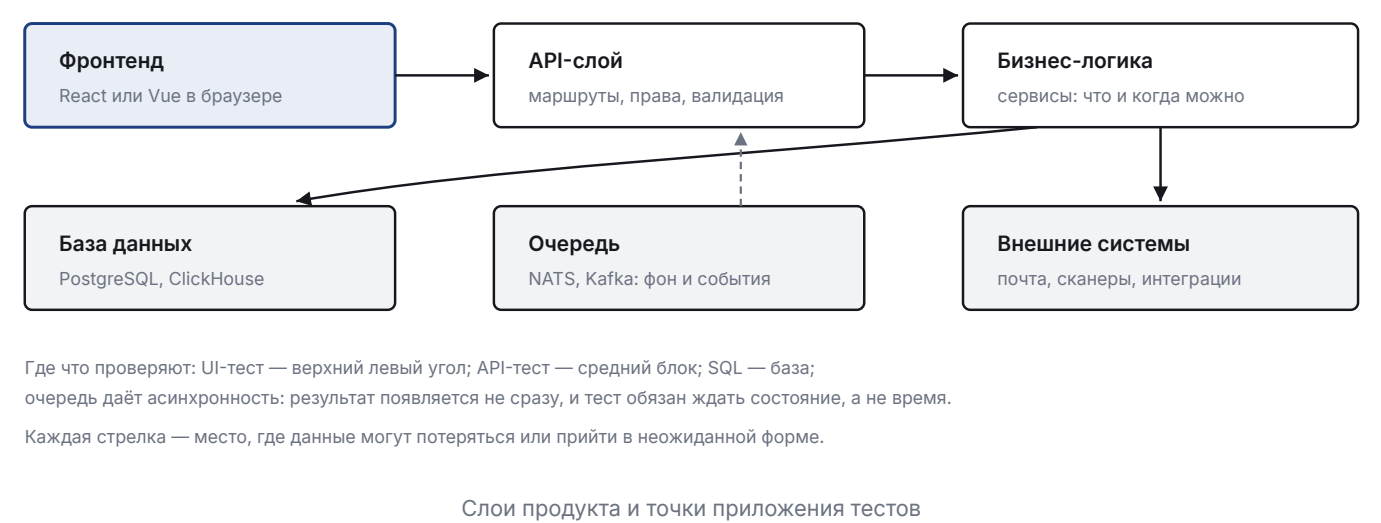
Что нового узнал про HTTP и HTTPS:

Что посмотрел в Network на реальном сайте:

6 Как устроен продукт изнутри

Чтобы понимать, где может сломаться, нужно представлять, из каких частей собран современный веб-продукт.

6.1 Слои



6.2 Монолит, микросервисы, микрофронтенды

Подход	Суть	Что меняется для тестировщика
Монолит	всё приложение — одна программа	проще стенд, но релиз целиком: регресс шире
Микросервисы	десятки маленьких сервисов общаются по сети	ошибки на стыках, частичные отказы, нужны контрактные тесты
Микрофронтенды	интерфейс собирается из частей разных команд	несогласованность версий и стилей, общая библиотека компонентов

В ПРОЕКТЕ SYSTEMBURO

R-Vision строит интерфейсы продуктов EVO как микрофронтенды поверх общей библиотеки компонентов, а бэкенд разбит на сервисы, часть которых переписана на Rust ради производительности. Отсюда и вакансия фронтендера про «системные компоненты для объединения микрофронтендов».

6.3 Асинхронность: почему результат появляется не сразу

Тяжёлые операции не делают в момент запроса. Сервер отвечает «принято» (код 202), кладёт задачу в очередь, а обработка идёт в фоне. Для тестировщика это означает: после действия результат может появиться через секунду, а может через минуту.

ЛОВУШКА

Проверка асинхронной операции через фиксированную паузу — источник флаков. Правильно: опрашивать состояние до появления результата с разумным пределом ожидания либо проверять факт постановки в очередь отдельно от факта выполнения.

6.4 Docker и Kubernetes простыми словами

- **Контейнер** — приложение вместе со всем, что ему нужно для запуска. Работает одинаково на ноутбуке и на сервере, чем убирает споры «у меня же запускается».
 - **Docker** — инструмент, который эти контейнеры собирает и запускает.
 - **Kubernetes** — система, которая держит много контейнеров на многих серверах: сама перезапускает упавшие, добавляет копии под нагрузкой, выкатывает новые версии.
- Тестировщику это нужно для двух вещей: поднять стенд одной командой и понимать, что «сервис перезапустился» — обычное явление, а не всегда дефект.

П9 Найдите слой

Для каждого дефекта укажите слой и уровень теста, который поймал бы его дешевле всего:

Дефект	Слой	Каким тестом ловится
Кнопка «Отправить» не блокируется, можно нажать дважды		
Пользователь чужой организации получает данные по прямому запросу к API		
Заявка сохраняется без обязательного поля		
Уведомление приходит через 20 минут вместо минуты		

Т6 Проверьте себя

1. Зачем продукту очередь, если можно сделать всё сразу в ответе на запрос?
2. Почему проверку прав доступа нельзя тестировать только через интерфейс?
3. Что означает код 202 в ответе?

ИТОГ РАЗДЕЛА

Продукт — это слои: интерфейс, API, логика, база, очередь, внешние системы. Дефект всегда живёт в конкретном слое, и задача тестировщика — назвать его. Асинхронность требует ожидания состояния, а не паузы.

МОИ ЗАМЕТКИ

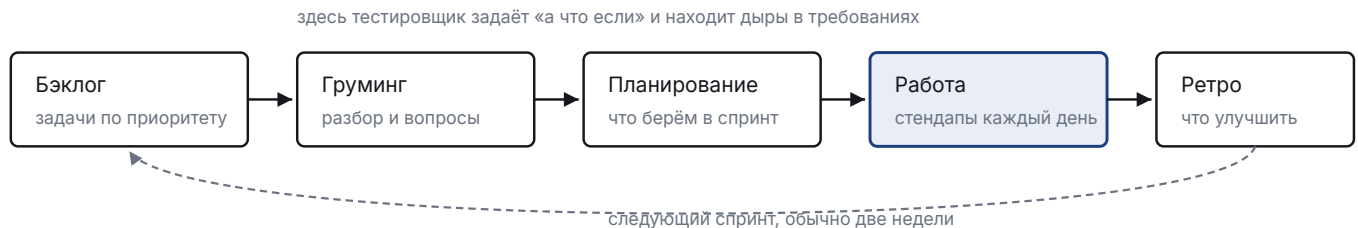
Слои продукта, который я знаю, и где там очередь:

Какие дефекты живут на стыках слоёв:

7 Как работает команда: процесс, Git, окружения

Стажёру важнее понимать поток работы, чем помнить термины. Но термины спросят, поэтому и то и другое.

7.1 Спринт и его ритуалы



Цикл спринта: где в нём место тестировщика

User story	задача словами пользователя: «как сотрудник, я хочу подать заявку, чтобы получить пропуск»
Критерии приёмки	список условий, при которых задача считается выполненной. Основа ваших тест-кейсов
Definition of Ready	когда задачу можно брать в работу: требования понятны, макет есть, критерии описаны
Definition of Done	когда задача закрыта: код в ветке, ревью пройдено, тесты зелёные, проверено тестировщиком

7.2 Git: как код попадает в продукт



Путь изменения: ветка, коммиты, пул-реквест, слияние

```
git clone <адрес>           скачать репозиторий
git checkout -b feature/tests создать свою ветку
git status                  что изменено
git add . && git commit -m "..." зафиксировать изменения
git push -u origin feature/tests отправить на сервер
git pull                   забрать чужие изменения
git log --oneline -5        последние коммиты
```

ЛОВУШКА

Работа напрямую в основной ветке и коммит «fix» без описания — два признака, по которым в команде сразу видно новичка. Сообщение коммита пишется так, чтобы через полгода было понятно, что и зачем менялось.

7.3 Окружения

Окружение	Кто пользуется	Что тестируют
dev	разработчики	черновые сборки, часто сломано, данных мало
test	тестировщики	основная работа: кейсы, регресс, автотесты
staging	команда и заказчик	копия боевого: финальная проверка перед релизом
prod	пользователи	только смоук после выката, эксперименты запрещены

Почему дефекты доходят до прода даже при зелёных тестах: на стенде другие данные и их объём, другие настройки, другая нагрузка, отключены внешние интеграции. Поэтому после выката делают смоук на бою, а рискованные функции прячут за **фича-флагом** — переключателем, которым функцию можно выключить без нового релиза.

В ПРОЕКТЕ SYSTEMBURO

Инцидент из истории systemburo: все проверки в CI были зелёными, а деплой падал четыре раза подряд и фронтенд на стенд не попадал. Вывод, ставший правилом проекта: после выката нужно своими глазами открыть стенд и дёрнуть реальный метод API, а не верить зелёному значку.

П10 Критерии приёмки

Дана user story. Допишите критерии приёмки так, чтобы по ним можно было писать тест-кейсы.

Как сотрудник организации, я хочу отозвать свою заявку, чтобы отменить визит, если он не состоится.

Критерии приёмки:

1. Кнопка «Отозвать» видна только автору заявки
2. _____
3. _____
4. После отзыва статус заявки становится «Отозвана»
5. _____ (про API)

П11 Последовательность Git

Вы дописали два автотеста в ret-проекте. Расставьте шаги по порядку и допишите пропущенные команды:

```
__ git push -u origin tests/application-form
__ git checkout -b tests/application-form
__ _____ (посмотреть, что изменено)
__ git commit -m "test: добавил проверки границ срока заявки"
__ _____ (добавить файлы в коммит)
```

Т7 Проверьте себя

1. Чем Definition of Ready отличается от Definition of Done?
2. На каком окружении нельзя ставить эксперименты и почему?
3. Зачем нужен фича-флаг?
4. Все тесты зелёные, релиз выкатили, продукт не работает. Как такое возможно? Назовите две причины.

ИТОГ РАЗДЕЛА

Задача идёт по кругу: бэклог, груминг, планирование, работа, ретро. Код едет через ветку и пул-реквест с ревью. Окружений несколько, и зелёный CI не заменяет проверку глазами после выката.

МОИ ЗАМЕТКИ

Как устроен процесс в команде, которую я видел:

Команды Git, которые надо довести до автоматизма:

8 Логи и мониторинг: доказательства для баг-репорта

Скриншот показывает симптом. Лог показывает причину. Разница между «что-то упало» и «упало вот здесь» — это лог.

8.1 Что такое лог

Лог — поток записей о том, что делала программа. У каждой записи есть время, уровень важности и сообщение, а в современных системах ещё и структурированные поля.

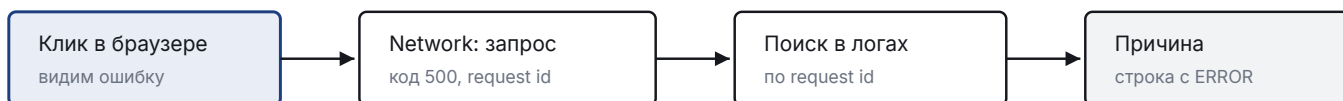
DEBUG	подробности для разработчика, в бою обычно выключены
INFO	обычные события: запрос обработан, задача выполнена
WARN	странно, но работаем дальше
ERROR	операция не выполнена. Именно это ищут после дефекта

```
2026-09-06T14:22:31Z ERROR request_id=7f3a9c user_id=12 method=POST
path=/api/applications status=500
msg="failed to create application: date_to before date_from"
```

Из одной строки видно: когда, кто, какой запрос, какой код и по какой причине. К баг-репорту достаточно приложить её вместе с `request_id`.

8.2 Как связать свой клик с записью в логе

Тот же путь работает и в обратную сторону: по ошибке в логе восстанавливают действие пользователя



Инструменты: Kibana или Grafana Loki для поиска по логам, Grafana с Prometheus для графиков и тревог

От симптома к причине через идентификатор запроса

8.3 Что прикладывать к баг-репорту

- Скриншот или короткое видео — что видел пользователь.
- Запрос из Network: «Сору as cURL» или выгрузка HAR.
- Ошибку из консоли браузера, если она есть.
- Строку лога сервера с идентификатором запроса и временем.
- Для автотеста — трейс Playwright: в нём уже есть и сеть, и консоль, и снимки страницы.

ЛОВУШКА

Время в логах обычно в UTC, а на экране у вас местное. Разница в три часа приводит к тому, что «нужной записи нет». Всегда сверяйте часовой пояс, прежде чем заявлять, что лог пустой.

П12 Прочитайте лог

```
2026-09-06T09:15:02Z INFO request_id=a1b2 user_id=12 POST /api/login 200
2026-09-06T09:15:44Z INFO request_id=c3d4 user_id=12 POST /api/applications 201
2026-09-06T09:15:44Z WARN request_id=c3d4 msg="attachment expires in 2 days"
2026-09-06T09:16:10Z ERROR request_id=e5f6 user_id=12 GET /api/applications/1042 500
msg="nil pointer dereference in buildApplicationResponse"
```

1. Что делал пользователь по шагам? _____
2. Какая запись — дефект, а какая просто предупреждение? ____
3. Что напишете в заголовке баг-репорта? _____
4. Какое поле поможет разработчику найти этот случай? _____

Т8 Проверьте себя

1. Чем WARN отличается от ERROR?
2. Зачем нужен идентификатор запроса?
3. В логе нет записи о вашем действии. Две причины, кроме «логи не пишутся».

ИТОГ РАЗДЕЛА

Лог превращает баг-репорт из жалобы в доказательство. Ищите строку с ERROR по идентификатору запроса, прикладывайте её вместе с запросом из Network и помните про часовые пояса.

МОИ ЗАМЕТКИ

Что нашёл в логе и как это связал с запросом:

Что теперь буду прикладывать к баг-репорту:

9 Домен: что тестируют в продуктах кибербезопасности

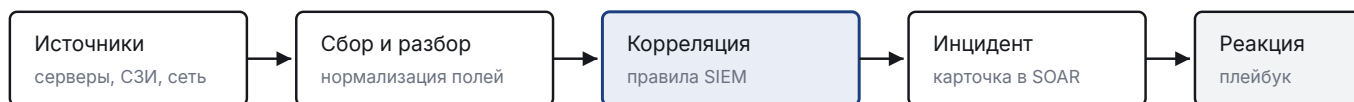
Словарь, без которого разговор на собеседовании в R-Vision будет неполным.

9.1 Словарь

SOC	подразделение, которое круглосуточно следит за безопасностью компании. Главный пользователь продуктов R-Vision
Событие	любая запись из системы: вход в систему, запуск процесса, срабатывание антивируса
Инцидент	событие или их цепочка, которая означает возможную атаку и требует реакции
Корреляция	правило, которое из потока событий собирает инцидент: «пять неудачных входов, затем удачный, с нового адреса»
Актив	то, что защищаем: сервер, рабочая станция, сервис
Уязвимость, CVE, CVSS	слабое место, его номер в мировом каталоге и оценка опасности от 0 до 10
Плейбук	сценарий реагирования: что система делает автоматически при инциденте

9.2 Как это складывается в продукт

10 000 событий в секунду — референсный поток для R-Vision SIEM; события хранятся в ClickHouse



Тестировщик проверяет каждый переход: не потерялось ли поле, сработало ли правило, созданся ли инцидент

Конвейер SIEM и SOAR: от события до реакции

9.3 Что это меняет в работе тестировщика

- **Объёмы.** Интерфейс должен оставаться живым на таблице в миллионы строк. Отсюда важность фильтров, пагинации и нагрузочных прогонов.
- **Права.** В ИБ-продуктах роли решают всё: аналитик SOC, администратор, аудитор видят разное. Проверка доступа идёт не только по интерфейсу, но и прямым запросом к API.
- **Аудит.** Система обязана фиксировать, кто что сделал. Пропавшая запись в журнале аудита — дефект, даже если функция работает.
- **Интеграции.** Продукт общается с чужими системами, и половина дефектов живёт на стыках: неожиданный формат, недоступный сервис, таймаут.

T9 Проверьте себя

1. Чем событие отличается от инцидента?
2. Что делает правило корреляции?
3. Почему в ИБ-продукте проверку прав нельзя ограничить интерфейсом?
4. Функция работает, но действие не попало в журнал аудита. Это дефект?

ИТОГ РАЗДЕЛА

SIEM собирает события и превращает их в инциденты, SOAR реагирует по плейбукам, VM закрывает уязвимости, TIP приносит данные об угрозах. Для тестировщика это означает большие объёмы, строгие права, обязательный аудит и много интеграций.

МОИ ЗАМЕТКИ

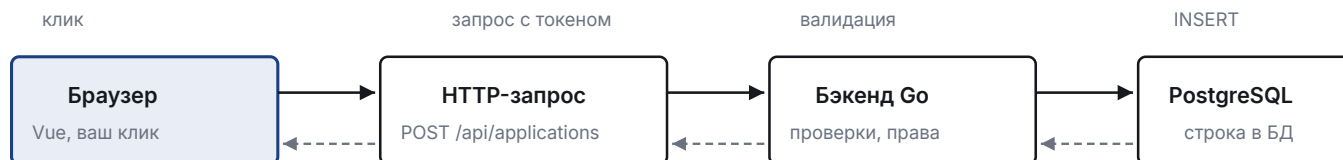
Термины домена, которые надо повторить:

Вопросы про продукты, которые задам на собеседовании:

10 Веб под капотом: DevTools, DOM, локаторы

Тестировщик, который видит только картинку на экране, находит половину дефектов. Вторая половина видна во вкладке *Network*.

10.1 Что происходит при нажатии кнопки



Уровни проверки:

UI-тест видит всю цепочку целиком, но медленно и хрупко

API-тест бьёт в середину: быстро, стабильно, без браузера. SQL-запрос проверяет самый конец

Один клик проходит четыре слоя. Дефект может жить в любом

Практический вывод: увидев на экране «Ошибка», первым делом открывайте *Network*. Три исхода дают три разных баг-репорта: запрос не ушёл вовсе (дефект фронта), ушёл и вернул 400 (дефект данных или валидации), вернул 500 (дефект бэка).

10.2 DevTools: две вкладки, которые нужны всегда

Вкладка	Что смотреть
Network	какой запрос ушёл, метод и URL, код ответа, тело запроса (Payload) и тело ответа (Response), заголовки, время
Console	красные ошибки JavaScript, предупреждения. Ошибка в консоли — почти всегда баг, даже если внешне всё нарисовалось

Полезные приёмы: фильтр `Fetch/XHR` убирает картинки и шрифты; галка `Preserve log` сохраняет запросы при переходах; правый клик по запросу → `Copy as cURL` даёт готовую команду для баг-репорта; `Save all as HAR` выгружает всю сессию файлом, который можно приложить к дефекту.

10.3 DOM: где живут элементы

Страница в браузере — дерево элементов. Автотест ищет узел этого дерева и действует над ним. Чем устойчивее признак поиска, тем меньше тест ломается от вёрстки.

```
<form data-testid="login-form">
  <input data-testid="login-input-username" class="lk-input" placeholder="Логин">
  <input data-testid="login-input-password" type="password" class="lk-input">
  <button data-testid="login-button-submit" class="lk-button">Войти</button>
  <p data-testid="login-error-message" class="error">Неверный логин или пароль</p>
</form>
```

В ПРОЕКТЕ SYSTEMBURO

Это реальная разметка формы входа systemburo: атрибуты `data-testid` проставлены специально для тестов. Их не меняет дизайнер при перекраске кнопки, поэтому тест на них не падает.

10.4 Локаторы

Тип	Пример	Когда
CSS по testid	<code>[data-testid="login-button-submit"]</code>	основной выбор
CSS по id	<code>#username</code>	если id стабильный
CSS по классу	<code>.lk-input</code>	крайний случай: класс меняет дизайнер
CSS вложенность	<code>form.login > input:first-child</code>	когда нет других признаков
XPath по тексту	<code>//button[contains(text(),'Войти')]</code>	текст важен по смыслу
XPath по оси	<code>//label[text()='Логин']/following-sibling::input</code>	элемент опознаётся через соседа

ЛОВУШКА

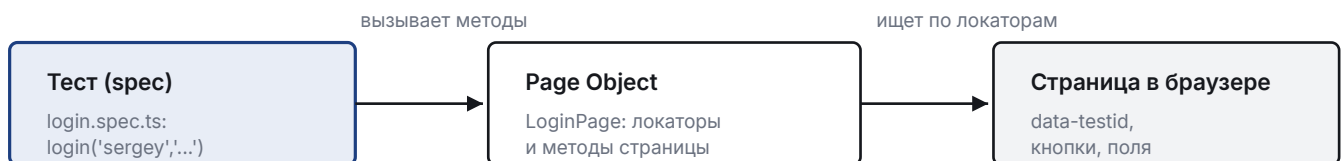
Абсолютный XPath вида `/html/body/div[2]/div/form/input[1]` — худший из возможных локаторов. Он ломается от любой обёртки, добавленной вёрсткой, и ничего не говорит читателю теста о том, что за элемент выбран.

Приоритет, принятый в командах

1. По роли и доступному имени: кнопка, поле, ссылка так, как их видит пользователь и скринридер.
2. По `data-testid`: договорённость команды, самый стабильный технический признак.
3. По тексту: если текст — часть смысла (например, сообщение об ошибке).
4. CSS-селектор от ближайшего устойчивого родителя.
5. XPath — только когда без него нельзя.

10.5 Page Object

Page Object (POM) — паттерн, при котором страница описывается классом: локаторы лежат в полях, действия — в методах, а тест вызывает методы и не знает ни одного селектора.



Поменялась вёрстка — правим один файл POM, а не двадцать тестов. В этом вся выгода.

Тест не знает о селекторах: между ним и страницей стоит Page Object

Так выглядит настоящий POM формы входа в systemburo (e2e/pages/LoginPage.cjs):

```
class LoginPage {
  constructor(page) {
    this.page = page;
    this.form = page.getByTestId('login-form');
    this.usernameInput = page.getByTestId('login-input-username');
    this.passwordInput = page.getByTestId('login-input-password');
    this.submitButton = page.getByTestId('login-button-submit');
    this.errorMessage = page.getByTestId('login-error-message');
  }

  async goto() { await this.page.goto('/'); }

  async login(username, password) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}
```

Пять локаторов, два действия, ни одного ассерта. Проверки живут в тестах.

П13 Соберите локаторы

Дан фрагмент разметки карточки заявки. Впишите локатор для каждого элемента, соблюдая приоритет из 5.4:

```
<article data-testid="application-card" class="card">
  <h3 class="card__title">Заявка № 1042</h3>
  <span data-testid="application-status" class="badge">Непрочитано</span>
  <button class="btn btn--danger">Отозвать</button>
</article>
```

Карточка целиком: _____
 Статус заявки: _____
 Кнопка «Отозвать»: _____
 Заголовок с номером (текст важен): _____

П14 Достройте Page Object

Дан каркас POM страницы подачи заявки. Допишите пропущенное: три локатора и один метод.

```
class CreateApplicationPage {
  constructor(page) {
    this.page = page;
    this.form = page.getByTestId('application-form');
    this.organizationSelect = page.getByTestId('application-organization');
    this.dateFrom = _____;
    this.dateTo = _____;
    this.submitButton = _____;
    this.fieldError = page.getByTestId('application-field-error');
  }

  async goto() {
    await this.page.goto('/applications/create');
  }

  // метод: заполнить форму и отправить
  async fillAndSubmit({ organization, from, to }) {
    _____
    _____
    _____
    _____
  }
}
```

Подсказка: у `select` в Playwright есть метод `selectOption`, у полей — `fill`, у кнопки — `click`.

T10 Проверьте себя

1. Кнопка «Отправить» не реагирует. Какую вкладку DevTools открыть первой и что там искать?
2. Почему `.btn--danger` — плохой локатор для кнопки «Отозвать»?
3. Что из этого не должно лежать в Page Object?
а) локаторы полей б) метод «заполнить форму» в) проверка `expect(...).toHaveText('Непрочитано')` внутри каждого метода
4. Тест упал после редизайна, хотя функция работает. Что это говорит о локаторах?

ИТОГ РАЗДЕЛА

Клик — это запрос, запрос — это код ответа, код ответа — это адрес дефекта. Локаторы выбираем по устойчивости: роль, `testid`, текст, и только потом CSS с XPath. Page Object держит селекторы в одном месте, чтобы редизайн стоил одной правки.

МОИ ЗАМЕТКИ

Локаторы, снятые с реальной страницы:

Что показал Network на моей странице:

11 API: проверка без браузера

Быстрее UI в десятки раз и стабильнее. В вакансии R-Vision автоматизация API-тестов стоит отдельной строкой требований.

11.1 HTTP за пять минут

Метод	Смысл	Пример
GET	получить, ничего не меняя	GET /api/applications
POST	создать	POST /api/applications
PUT / PATCH	заменить целиком / изменить частично	PATCH /api/applications/42
DELETE	удалить	DELETE /api/cars/7

Код	Значение	Что это значит для тестировщика
200 / 201	успех / создано	проверяем тело ответа, а не только код
204	успех без тела	типично для DELETE
400	плохой запрос	клиент прислал мусор: не тот тип, нет поля
401	не аутентифицирован	токена нет или он протух
403	нет прав	пользователь известен, но доступ запрещён
404	не найдено	иногда используется вместо 403, чтобы скрыть существование объекта
409	конфликт	дубль, нарушение уникальности
422	не прошло валидацию	формат верный, значения недопустимы
500	ошибка сервера	всегда дефект, даже если данные были странные

ЛОВУШКА

Различие 401 и 403 спрашивают почти на каждом собеседовании. 401 — «я не знаю, кто ты»; 403 — «я знаю, кто ты, и тебе нельзя».

11.2 Тело ответа и конверт

Большинство API оборачивают полезные данные в конверт: успех, данные, метаданные, ошибка. Тестировщик обязан знать форму конверта своего продукта, иначе будет проверять не то поле.

В ПРОЕКТЕ SYSTEMBURO

В systemburo конверт задан в коде и одинаков для всех методов:

```
type Response struct {
    Success bool   `json:"success"`
    Data    any    `json:"data,omitempty"`
    Meta    any    `json:"meta,omitempty"`
    Error   string `json:"error,omitempty"`
}
```

Ответ на список выглядит так:

```
{
  "success": true,
  "data": [ { "id": 1042, "status": "Непрочитано" } ],
  "meta": { "page": 1, "per_page": 20, "total": 137 }
}
```

А ошибка — так: `{"success": false, "error": "Invalid ID"}`.

ЛОВУШКА

Отдельная ловушка того же проекта: часть методов кладёт объект ещё в одну обёртку (`data.table` вместо `data`). Из-за этого интерфейс однажды не показал таблицу, хотя данные приходили. Мораль: перед написанием проверки посмотрите настоящий ответ в Network, а не предполагайте его форму.

11.3 Авторизация

Обычная схема: логин отдаёт токен, дальше он ходит в заголовке `Authorization: Bearer <токен>`. Токен (JWT) состоит из трёх частей через точку и живёт ограниченное время. Что проверять тестировщику: запрос без токена даёт 401, с чужим токеном 403 или 404, с протухшим — 401, а после выхода из системы старый токен перестаёт работать.

11.4 Как дёргать API руками

```
curl -s -X POST http://localhost:8080/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"sergey","password":"secret"}'

curl -s http://localhost:8080/api/applications \
  -H "Authorization: Bearer $TOKEN"
```

То же самое в Postman: коллекция запросов, переменная окружения `{{token}}`, вкладка Tests для простых проверок. Для баг-репорта удобнее всего копировать запрос прямо из DevTools через «Copy as cURL» — разработчик выполнит его у себя.

В ПРОЕКТЕ SYSTEMBURO

В E2E-наборе systemburo API-хелпер устроен ровно так: получает токен через `/login`, подставляет `Authorization: Bearer` и разворачивает конверт функцией `unwrap(body)`, возвращающей `body.data`.

11.5 REST и GraphQL

	REST	GraphQL
Адреса	много путей: <code>/applications</code> , <code>/users</code>	один путь <code>/graphql</code>
Выбор полей	сервер решает, что вернуть	клиент перечисляет нужные поля в запросе
Ошибки	кодом HTTP	часто код 200, а ошибка внутри тела в поле <code>errors</code>

ЛОВУШКА

Последняя строка — ловушка для тестировщика на проектах R-Vision, где GraphQL используется: проверка «ответ 200, значит всё хорошо» пропустит ошибку. Смотреть надо в тело.

П15 Разберите ответ

Дан ответ сервера. Ответьте на вопросы под ним:

```
HTTP/1.1 200 OK
{
  "success": true,
  "data": { "id": 1042, "status": "Непрочитано", "organization_id": 7 },
  "meta": null
}
```

1. Какой код ответа ожидался бы для успешного СОЗДАНИЯ заявки? _____
2. Заявка создана, но в списке пользователя не видна. В каком слое искать дефект в первую очередь и почему? _____
3. Напишите проверку на JavaScript, которая убеждается, что статус новой заявки именно «Непрочитано»:
`expect(_____.toBe(_____));`

П16 Допишите API-проверку

Заготовка теста на Playwright. Впишите пропущенное:

```
test('создание заявки возвращает 201 и заявку в статусе Непрочитано',
  async ({ request }) => {
    const token = await getToken('sergey', 'secret');

    const res = await request.post('/api/applications', {
      headers: { Authorization: `Bearer ${token}` },
      data: { organization_id: 7, date_from: '2026-09-06',
        date_to: '2026-10-05' },
    });

    expect(res.status()).toBe(______);
    const body = await res.json();
    expect(body.success).toBe(______);
    expect(body.data.status).toBe('_____');
  });
```

Затем допишите второй тест: тот же запрос без заголовка **Authorization** должен вернуть 401. Три строки.

Т11 Проверьте себя

1. Запрос вернул 403. Токен валидный. В чём причина?
2. Что быстрее и стабильнее для проверки бизнес-правила «срок не больше 30 дней»: UI-тест или API-тест? Почему?
3. Сервер вернул 200, но в теле `{"success": false, "error": "..."} . Это дефект?
а) нет, раз код 200 б) да, если по контракту ошибка должна отдаваться кодом 4xx в) зависит от контракта API — надо свериться с документацией`
4. Зачем тестировщику копировать запрос как cURL?

ИТОГ РАЗДЕЛА

API-проверка бьёт в середину цепочки: без браузера, быстро, стабильно. Знать надо методы, коды, заголовок авторизации и форму конверта своего продукта. Настоящую форму ответа всегда смотрим в Network, а не додумываем.

МОИ ЗАМЕТКИ

Коды ответов, которые встретил сам:

Форма конверта в API, который я смотрел:

12 SQL: проверка того, что реально сохранилось

Интерфейс может показать «Успешно» и не сохранить ничего. База — последняя инстанция.

12.1 Зачем это тестировщику

- Убедиться, что после действия в интерфейсе строка появилась и поля заполнены верно.
- Подготовить данные для проверки: найти заявку в нужном статусе, пользователя с нужной ролью.
- Понять причину дефекта: в базе пусто или лежит не то, что показывает экран.

В ПРОЕКТЕ SYSTEMBURO

Упрощённая схема systemburo, на которой мы дальше пишем запросы:

```
users          (id, username, organization_id, role_id, is_active)
organizations  (id, name, type)
applications    (id, application_number, status, organization_id,
                 sender_user_id, sending_datetime, accepted_at)
employees      (id, attachment_id, last_name, first_name)
attachments    (id, application_id, date_from, date_to)
```

12.2 Минимум конструкций

```
-- выбрать колонки с условием
SELECT id, status, sending_datetime
FROM applications
WHERE organization_id = 7 AND status = 'Непрочитано'
ORDER BY sending_datetime DESC
LIMIT 10;

-- соединить с другой таблицей
SELECT a.id, a.status, u.username, o.name AS organization
FROM applications a
JOIN users u ON u.id = a.sender_user_id
JOIN organizations o ON o.id = a.organization_id
WHERE a.id = 1042;

-- посчитать по группам
SELECT status, COUNT(*) AS cnt
FROM applications
GROUP BY status
HAVING COUNT(*) > 5
ORDER BY cnt DESC;
```

Конструкция	Что делает
WHERE	фильтрует строки до группировки
JOIN (INNER)	оставляет только строки, у которых есть пара в обеих таблицах
LEFT JOIN	оставляет все строки левой таблицы, справа NULL если пары нет
GROUP BY + COUNT	считает по группам
HAVING	фильтрует уже посчитанные группы
IS NULL	единственный способ сравнить с NULL: <code>= NULL</code> не работает

ЛОВУШКА

Классическая ловушка на собеседовании: `LEFT JOIN` с условием по правой таблице в `WHERE` превращается в обычный `INNER JOIN`, потому что строки с `NULL` отсеиваются. Условие для правой таблицы надо писать в `ON`.

12.3 Типовые проверки после действия в интерфейсе

```
-- 1. заявка действительно создана и в правильном статусе
SELECT id, status FROM applications
WHERE sender_user_id = 12
ORDER BY id DESC LIMIT 1;

-- 2. двойной клик не создал дубль
SELECT COUNT(*) FROM applications
WHERE sender_user_id = 12
  AND sending_datetime > now() - interval '1 minute';

-- 3. у заявки сохранились все сотрудники из формы
SELECT e.last_name, e.first_name
FROM employees e
JOIN attachments att ON att.id = e.attachment_id
WHERE att.application_id = 1042;
```

П17 Допишите запросы

```
-- а) сколько заявок в каждом статусе у организации 7
SELECT _____, COUNT(*) AS cnt
FROM applications
WHERE _____
GROUP BY _____;

-- б) пользователи, которые ни разу не подавали заявку
-- (подсказка: LEFT JOIN и проверка на NULL)
SELECT u.username
FROM users u
_____ JOIN applications a ON a.sender_user_id = u.id
WHERE a.id _____;

-- в) заявки, принятые в работу, но без даты принятия –
-- признак дефекта в бэкенде
SELECT id FROM applications
WHERE status = '_____' AND accepted_at _____;
```

Т12 Проверьте себя

1. Чем WHERE отличается от HAVING ?
2. Как найти строки, у которых поле пустое?
а) WHERE col = NULL б) WHERE col IS NULL в) WHERE col = ''
3. Интерфейс показал «Заявка отправлена», в таблице applications новой строки нет. Ваши действия и какой severity?

ИТОГ РАЗДЕЛА

Тестировщику хватает SELECT, WHERE, JOIN, GROUP BY и IS NULL. Этого достаточно, чтобы отличить «интерфейс соврал» от «бэкенд не сохранил» и написать точный баг-репорт вместо «что-то не работает».

МОИ ЗАМЕТКИ

Запросы, которыегодились:

Что не сошлось между интерфейсом и базой:

13 TypeScript ровно столько, сколько нужно тесту

Не курс по языку. Только те конструкции, без которых не написать и не прочитать автотест на Playwright.

13.1 Зачем тесту типы

TypeScript — это JavaScript, которому дописали типы. Пользы для тестировщика две: редактор подсказывает методы (набрали `page` и видите весь список), а опечатка ловится до запуска, а не на десятой минуте прогона.

```
let count: number = 5;
let title: string = 'Заявка № 1042';
let isActive: boolean = true;
let statuses: string[] = ['Непрочитано', 'В обработке'];
```

13.2 Интерфейс: форма объекта

```
interface Application {
  id: number;
  status: string;
  organizationId: number;
  acceptedAt?: string; // знак ? — поле необязательное, может отсутствовать
}

const app: Application = { id: 1042, status: 'Непрочитано', organizationId: 7 };
```

В тестах интерфейсы удобны для описания тестовых данных и ответов API: если бэкенд переименует поле, редактор подсветит все места, где оно используется.

13.3 Функции

```
// обычная функция с типами параметров и возврата
function daysBetween(from: string, to: string): number {
  return (new Date(to).getTime() - new Date(from).getTime()) / 86400000;
}

// стрелочная функция — тот же смысл, короче запись
const isLongStay = (days: number): boolean => days > 30;
```

13.4 async, await и промисы: главное для Playwright

Промис (Promise) — обещание результата, который придёт позже: ответ сервера, загрузка страницы, клик по кнопке. **await** — «подожди этот результат и продолжай». Функция, внутри которой есть `await`, объявляется как `async`.

```
test('вход в систему', async ({ page }) => {
  await page.goto('/');
  await page.getByTestId('login-input-username').fill('sergey');
  await page.getByTestId('login-button-submit').click();
});
```

ЛОВУШКА

Забытый `await` — ошибка номер один у новичков в Playwright. Тест не падает, а завершается раньше, чем действие выполнено: клик «уходит в пустоту», проверка срабатывает на старой странице, а результат прогона получается случайным. Правило простое: почти каждая строка теста на Playwright начинается с `await`.

13.5 Класс: основа Page Object

```
import { Page, Locator } from 'playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly submitButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.getByTestId('login-input-username');
    this.submitButton = page.getByTestId('login-button-submit');
  }

  async login(username: string, password: string): Promise<void> {
    await this.usernameInput.fill(username);
    await this.page.getByTestId('login-input-password').fill(password);
    await this.submitButton.click();
  }
}
```

`readonly` означает «поле задаётся один раз в конструкторе и дальше не меняется» — ровно то, что нужно локаторам. `export` делает класс доступным другим файлам.

13.6 Импорт и экспорт

```
// файл pages/LoginPage.ts
export class LoginPage { /* ... */ }

// файл tests/login.spec.ts
import { test, expect } from 'playwright/test';
import { LoginPage } from '../pages/LoginPage';
```

П18 Найдите три ошибки

В коде три дефекта: один типовой, два по части `async`. Найдите и исправьте.

```
test('отправка заявки', ({ page }) => {
  const form = new CreateApplicationPage(page);
  form.goto();
  await form.fillAndSubmit({ organization: 7, from: '2026-09-06',
    to: '2026-10-05' });

  const count: string = 1;
  expect(page.getByTestId('application-status')).toHaveText('Непрочитано');
});
```

П19 Опишите данные типами

Допишите интерфейс под ответ API из раздела 11 и типизируйте функцию:

```
interface ApiResponse<T> {
  success: _____;
  data: _____;
  error?: _____;
}

// функция должна возвращать данные из конверта
function unwrap<T>(body: ApiResponse<T>): _____ {
  return _____;
}
```

ИТОГ РАЗДЕЛА

Для автотестов достаточно: типы переменных, интерфейсы для данных, функции, классы для POM, импорт-экспорт и, главное, `async` с `await`. Всё остальное в языке подождёт до первой рабочей задачи.

МОИ ЗАМЕТКИ

Конструкции TypeScript, которые пока не даются:

Ошибки с `await`, которые я поймал:

14 Playwright: автотесты, которые не разваливаются

Инструмент, который в вакансии значит «плюсом», а на деле является входным билетом: у QA Automation в R-Vision он в обязательных требованиях.

14.1 Установка и структура проекта

```
npm init playwrightlatest
```

```
qa-workbook/
├── e2e/
│   ├── pages/      Page Object'ы
│   ├── fixtures/   свои фикстуры
│   └── tests/       спеки: *.spec.ts
├── playwright.config.ts  конфиг: адрес стенда, таймауты, репортеры
└── package.json
```

В ПРОЕКТЕ SYSTEMBURO

Конфиг systemburo устроен так же и показывает минимально нужный набор настроек:

```
module.exports = defineConfig({
  testDir: './e2e/tests',
  timeout: 30000,
  retries: 1,
  use: {
    baseUrl: process.env.E2E_BASE_URL || 'http://localhost:8081',
    headless: true,
    screenshot: 'only-on-failure',
    trace: 'retain-on-failure',
  },
  projects: [{ name: 'chromium', use: { browserName: 'chromium' } }],
});
```

Обратите внимание на две строки: скриншот снимается только при падении, трейс сохраняется только для упавших. Это готовые вложения для баг-репорта.

14.2 Первый тест

```
import { test, expect } from 'playwright/test';

test('неверный пароль показывает ошибку', async ({ page }) => {
  await page.goto('/');
  await page.getByTestId('login-input-username').fill('sergey');
  await page.getByTestId('login-input-password').fill('wrong');
  await page.getByTestId('login-button-submit').click();

  await expect(page.getByTestId('login-error-message'))
    .toHaveText('Неверный логин или пароль');
});
```

14.3 Локаторы Playwright

Метод	Когда использовать
<code>getByRole('button', {name: 'Войти'})</code>	первый выбор: ищет так, как видит пользователь
<code>getByTestId('login-form')</code>	второй выбор: договорённость команды
<code>getByText('Непрочитано')</code>	когда важен именно текст
<code>getByLabel('Логин')</code>	поля формы с подписью
<code>locator('.card').first()</code>	крайний случай, для контейнеров
<code>card.getByRole('button')</code>	поиск внутри контейнера, а не по всей странице

ЛОВУШКА

Не связывайте локаторы через `.or()` : цепочка «или тот, или этот» матчит несколько элементов и делает падения непредсказуемыми. Один элемент — один точный локатор.

14.4 Автоожидания: почему в тестах нет `sleep`

Playwright сам ждёт, пока элемент появится, станет видимым и кликабельным, и только потом действует. Ожидание встроено в каждый метод и в каждый `expect`. Поэтому пауз в коде быть не должно.

```
// плохо: гадаем, хватит ли двух секунд
await page.waitForTimeout(2000);
await page.getByTestId('application-status').click();

// хорошо: ждём ровно того, что нужно, столько, сколько нужно
await expect(page.getByTestId('application-status')).toBeVisible();
await page.getByTestId('application-status').click();
```

ЛОВУШКА

Второй антипаттерн той же природы — `waitForLoadState('networkidle')`. На странице с опросами сервера или live-обновлениями сеть не «затихает» никогда, и тест виснет до таймаута. Ждать надо конкретный элемент.

14.5 Проверки

```
await expect(page.getByTestId('application-status')).toHaveText('Непрочитано');
await expect(page.getByTestId('application-card')).toHaveCount(3);
await expect(page.getByTestId('login-form')).toBeVisible();
await expect(page).toHaveURL(/\/applications\/\d+/);
await expect(page.getByTestId('submit')).toBeDisabled();

// мягкая проверка: тест продолжится, а падение зафиксировается
await expect.soft(page.getByTestId('title')).toHaveText('Заявка № 1042');

// повтор действия до успеха, для редких гонок
await expect(async () => {
  const count = await page.getByTestId('row').count();
  expect(count).toBeGreaterThan(0);
}).toPass();
```

14.6 Фикстуры: подготовка без копипасты

Фикстура — кусок подготовки, который Playwright выполняет за вас перед тестом и убирает после. Через неё дают готовый POM, авторизованную сессию, тестовые данные.

```
import { test as base } from 'playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { CreateApplicationPage } from '../pages/CreateApplicationPage';

export const test = base.extend<{
  loginPage: LoginPage;
  applicationPage: CreateApplicationPage;
}>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page)); // просто объект, без навигации
  },

  applicationPage: async ({ page, loginPage }, use) => {
    await loginPage.goto();
    await loginPage.login('sergey', 'secret'); // вошли
    const app = new CreateApplicationPage(page);
    await app.goto(); // открыли форму
    await use(app); // отдали тесту готовое состояние
  },
});

export { expect } from 'playwright/test';
```

Дальше тест импортирует `test` из своего файла фикстур, а не из библиотеки, и получает страницу уже в рабочем состоянии.

14.7 Тесты API внутри Playwright

```
test('без токена список заявок недоступен', async ({ request }) => {
  const res = await request.get('/api/applications');
  expect(res.status()).toBe(401);
});
```

Тот же `request` удобно использовать для подготовки данных: создать заявку запросом за доли секунды, а браузером проверять только то, ради чего тест написан.

14.8 Моки: тест без живого бэкенда

```
// подменяем ответ сервера
await page.route('**/api/applications', route => route.fulfill({
  status: 200,
  body: JSON.stringify({ success: true, data: [] }),
}));

// имитируем обрыв сети
await page.route('**/api/applications', route => route.abort());
```

Более честный способ — записать реальные ответы в HAR-файл и воспроизводить их: `await page.routeFromHAR('fixtures/applications.har')`. Такой мок ближе к реальности, чем руками написанный JSON, и его дифф видно в ревью.

14.9 Разбор упавшего теста

Инструмент	Что даёт
<code>npx playwright test --headed</code>	прогон с видимым браузером
<code>--debug</code>	пошаговое выполнение с инспектором
<code>npx playwright show-report</code>	HTML-отчёт с шагами и ошибками
<code>npx playwright show-trace trace.zip</code>	таймлайн: снимок DOM на каждом шаге, сеть, консоль
<code>npx playwright codegen <url></code>	запись действий в код: черновик, который потом переписывают руками

Трейс — главный инструмент разбора: по нему видно состояние страницы ровно в момент падения, и часто становится ясно, что тест ждал не тот элемент.

14.10 Антипаттерны, которые видно на собеседовании

Плохо	Как правильно
<code>waitForTimeout(3000)</code>	ожидание конкретного элемента через <code>expect</code>
Селекторы прямо в тесте	Page Object
Один тест на десять несвязанных проверок	один тест — одно поведение
Тесты зависят от порядка запуска	каждый тест готовит свои данные
Общее состояние между тестами	изоляция через фикстуры
Цепочка действий и один ассерт в конце	проверка после каждого значимого шага
Абсолютный XPath	роль или <code>testid</code>

П20 Допишите тест целиком

Заготовка теста на подачу заявки. Заполните пропуски так, чтобы проверка была по шагам, а не одним ассертом в конце.

```
import { test, expect } from '../fixtures/app';

test('заявка на 30 дней создаётся и попадает в список', async ({
  applicationPage, page }) => {

  await applicationPage.fillAndSubmit({
    organization: 'Ромашка',
    from: '2026-09-06',
    to: '2026-10-05',
  });

  // 1. форма закрылась
  await expect(applicationPage.form)._____

  // 2. появилось уведомление об успехе
  await expect(page.getByTestId('notification'))._____

  // 3. в списке появилась карточка со статусом «Непрочитано»
  const card = page.getByTestId('application-card').first();
  await expect(card.getByTestId('application-status'))
    ._____

  // 4. адрес страницы сменился на список заявок
  await expect(page)._____
});
```

П21 Почините нестабильный тест

Тест падает примерно в одном прогоне из пяти. Найдите три причины и перепишите.

```
test('фильтр по статусу', async ({ page }) => {
  await page.goto('/applications');
  await page.waitForTimeout(1500);
  await page.locator('div.filters > div:nth-child(2) > button').click();
  await page.waitForLoadState('networkidle');
  const rows = await page.locator('.row').count();
  expect(rows).toBe(7);
});
```

Подсказки: две паузы и один локатор здесь неправильные, а число 7 зависит от чужих данных.

Т13 Проверьте себя

1. Почему в Playwright не нужен `sleep`?
2. Что даёт трейс, чего не даёт скриншот?
3. Какой локатор предпочтительнее?
 - а) `page.locator('div.card > button.btn--primary')`
 - б) `page.getByRole('button', { name: 'Отправить' })`
 - в) `page.locator('//div[3]/button')`
4. Зачем нужна фикстура, если можно вызвать логин в начале каждого теста?
5. Тест проверяет создание заявки. Данные для него лучше готовить через интерфейс или через API? Назовите по одному аргументу за каждый вариант.

ИТОГ РАЗДЕЛА

Playwright сам ждёт — паузы не нужны. Локаторы: роль, `testid`, текст. Селекторы живут в Page Object, подготовка в фикстурах, разбор падений по трейсу. Эти пять правил отличают поддерживаемый набор тестов от свалки, которую через полгода выключают в CI.

МОИ ЗАМЕТКИ

Команды Playwright, которые использую чаще всего:

Почему упал мой тест и что помогло разобраться:

15 Где тесты живут дальше: CI, отчёты, флаки

Автотест, который запускают только руками, бесполезен. Ценность появляется, когда он гоняется на каждый пул-реквест.

15.1 Конвейер



Порядок этапов: дешёвые проверки раньше дорогих

В ПРОЕКТЕ SYSTEMBURO

В systemburo E2E запускаются на каждый пул-реквест в ветку dev и разбиты на четыре параллельных шарда с таймаутом 20 минут: рядом поднимаются PostgreSQL и бэкенд, задаются тестовые логины и повышаются лимиты частоты запросов, чтобы четыре потока не упёрлись в защиту от перебора. Это типичное устройство прогона, которое вы увидите и в GitLab CI у R-Vision.

15.2 Флаки

Флак (flaky test) — тест, который на одном и том же коде то падает, то проходит. Опаснее упавшего: команда привыкает перезапускать прогон и перестаёт читать красное.

Причина	Лечение
Пауза вместо ожидания элемента	убрать <code>waitForTimeout</code>
Тест зависит от данных другого теста	своя подготовка данных в фикстуре
Жёсткое ожидаемое число строк	проверять свою запись, а не общее количество
Гонка запросов, анимация	<code>expect(...).toPass()</code> или ожидание конечного состояния
Общая тестовая база на параллельных прогонах	изоляция данных, уникальные имена

15.3 Отчёты и TestOps

Результат прогона попадает в отчёт: HTML-репорт Playwright локально, Allure или Allure TestOps в компании. Что там смотрит тестировщик: какие тесты упали, на каком шаге, скриншот и трейс, история — падал ли этот тест раньше, и связан ли он с тест-кейсом из TMS.

T14 Проверьте себя

1. Почему юнит-тесты в конвейере идут раньше E2E?
2. Тест падает раз в пять прогонов. Ваши действия: перезапустить, отключить, разобраться?
3. Что опаснее для команды: стабильно красный тест или флак? Обоснуйте одним предложением.

ИТОГ РАЗДЕЛА

Тесты живут в конвейере: дешёвые раньше дорогих, падение блокирует движение дальше. Флак нужно чинить, а не перезапускать: доверие к прогону — единственная причина, по которой автотесты вообще держат.

МОИ ЗАМЕТКИ

Как устроен прогон в проекте, который я смотрел:

Флаки, которые встретил, и их причина:

16 Сборка: рет-проект, который показывают на собеседовании

Всё, что вы делали по ходу тетради, складывается в один репозиторий. Он и есть доказательство навыка вместо слов «изучал тестирование».

16.1 Что должно получиться

```
qa-workbook/
├── README.md           что за проект, что проверяется, как запустить
├── docs/
│   ├── checklist.md    чек-лист формы подачи (П6)
│   ├── test-cases.md   6 тест-кейсов
│   └── bug-reports.md  2 баг-репорта по образцу из раздела 4
├── e2e/
│   ├── pages/
│   │   ├── LoginPage.ts
│   │   └── CreateApplicationPage.ts
│   ├── fixtures/app.ts  фикстуры: bare POM и авторизованная
│   └── tests/
│       ├── login.spec.ts    негативный вход
│       ├── application.spec.ts  подача заявки, границы срока
│       └── api.spec.ts       создание через API + 401 без токена
├── sql/checks.sql       запросы проверки из раздела 12
└── playwright.config.ts
```

16.2 Порядок сборки

1. Создать репозиторий на GitHub, клонировать, `npm init playwrightlatest`.
2. Перенести чек-лист и кейсы в `docs/`. Это первые коммиты: тестировщик начинает с документов, а не с кода.
3. Написать `LoginPage`, на нём — первый тест на негативный вход.
4. Добавить `CreateApplicationPage` и фикстуру с авторизацией.
5. Написать три UI-теста: успешная подача, граница 30 дней, отклонение 31 дня.
6. Добавить API-тесты: создание и 401 без токена.
7. Положить SQL-проверки и описать в README, что именно они доказывают.
8. Прогнать всё, приложить в README вывод `npm playwright test` и скриншот отчёта.

ЛОВУШКА

Не берите чужой сайт-однодневку для тестов: он поменяется, и проект перестанет запускаться прямо на собеседовании. Подойдёт любой стабильный тренировочный стенд или локально поднятое приложение. Если гоняете на `systembugo` — поднимите его локально, а адрес вынесите в переменную окружения.

16.3 README, который читают

```
# QA workbook

Учебный тест-набор для формы подачи заявки.

## Что проверяется
- вход в систему: негативный сценарий
- подача заявки: успешный путь и границы срока (30 / 31 день)
- API: создание заявки, отказ без токена

## Стек
Playwright, TypeScript, Page Object, фикстуры

## Запуск
npm ci
npm playwright install --with-deps
npm playwright test

## Структура
docs/      ручные артефакты: чек-лист, кейсы, баг-репорты
e2e/       автотесты
sql/       проверки данных
```

16.4 Критерии готовности

Признак	Есть?
Селекторов в тестах нет, все в Page Object	
Ни одного <code>waitForTimeout</code>	
Каждый тест проходит в одиночку и в любом порядке	
Есть и позитивный, и негативный сценарий	
Есть API-тест, а не только UI	
История коммитов показывает ход работы, а не один коммит «all»	
README запускается по шагам на чужой машине	

П22 Финальная сборка

Соберите репозиторий по структуре выше. Минимальный набор для показа: 3 UI-теста, 2 API-теста, 2 Page Object, 1 фикстура, чек-лист, 6 кейсов, 2 баг-репорта, README. Ссылку на репозиторий положите в резюме отдельной строкой.

МОИ ЗАМЕТКИ

Что осталось доделать в pet-проекте:

Ссылка на репозиторий и дата первого прогона:

17 Собеседование: карта требований и вопросы

17.1 Требования вакансии и где они закрыты

Строка из вакансии R-Vision	Раздел тетради
Базовые знания программирования	13
Опыт разработки учебных или pet-проектов	16
Базовое понимание автоматизации тестирования	2, 14
Знание Page Object и локаторов	10, 14
Опыт работы с Git	11
Понимание методологий и пирамиды тестирования	2
Тест-дизайн, структура тест-кейса и баг-репорта	3, 4
Понимание SDLC и роли тестировщика	1, 2
Основы API: структура запросов и ответов, JSON	11
Навыки SQL	12
Плюсом: TypeScript и Playwright	13, 14
Задачи: DevTools, тест-кейсы, баг-репорты, анализ прогонов, Confluence	4, 10, 15

17.2 Вопросы, которые задают почти всегда

Вопрос	Каркас ответа
Чем отличается severity от priority	серьёзность объективна и моя, приоритет решает бизнес; пример с опечаткой на главной
Что такое пирамида тестирования	много юнитов, меньше интеграционных, мало E2E; почему перевёрнутая плоха
Как протестируете поле ввода даты	классы эквивалентности, границы, формат, пустое, прошедшая дата, часовой пояс
Разница 401 и 403	не аутентифицирован против нет прав
Что делать, если баг не воспроизводится у разработчика	уточнить окружение, роль, данные, приложить HAR и видео, повторить вместе
Что такое Page Object и зачем	селекторы в одном месте, редизайн стоит одной правки
Почему нельзя sleep в автотестах	гадание по времени, флаки, есть автоожидания
Тест упал в CI, что делаете	смотрю отчёт и трейс, воспроизвожу локально, отличаю дефект продукта от дефекта теста
Как проверите, что заявка сохранилась	UI, ответ API, строка в базе — три уровня

17.3 Что спросить самому

- Как оформляется стажировка: трудовой договор, ГПХ или ученический, и какая оплата.
- По каким критериям в конце шести месяцев принимают решение о переходе в штат.
- Насколько жёсткие 30-40 часов в неделю и возможна ли нижняя граница.
- На каком продукте буду работать и какой там стек автоматизации сейчас.
- Кто ментор и как устроена обратная связь.

В ПРОЕКТЕ SYSTEMBURO

Полезный контекст для разговора: у компании отдельная группа автоматизации из 10+ человек, автотесты на Playwright с TypeScript, отчёты в Allure TestOps, сборка в GitLab CI. Вопрос «на каком продукте и какой там фреймворк» показывает, что вы читали не только заголовок вакансии.

ИТОГ РАЗДЕЛА

На собеседовании стажёра проверяют не объём знаний, а способность рассуждать: как вы разложите незнакомую форму на проверки и как объясните найденный дефект. Тетрадь даёт словарь и практику для обоих вопросов, рет-проект — доказательство.

МОИ ЗАМЕТКИ

Вопросы, которые задам на собеседовании:

Что ответил неуверенно и надо повторить:

А Ответы и решения

Раздел 1

T1. 1) Нет: тестирование показывает наличие дефектов, но не доказывает их отсутствие. 2) AQA. 3) вариант б — искать в требованиях пропуски и противоречия до начала разработки.

Раздел 2

П1. Дата на 31 день вперёд отклоняется — функциональная проверка границы, severity значительный (нарушено бизнес-правило, но система работает). После правки календаря прогон старых сценариев — регресс, severity зависит от того, что сломалось; сама задача проверки критична перед релизом. Неоднозначность в требовании — статическое тестирование, дефекта ещё нет, это вопрос аналитику; severity не ставится, заводится уточнение к требованию.

T2. 1) Ошибка (человеческое действие), её результат в коде — дефект. 2) Продакт или лид. 3) б — смоук. 4) E2E медленные, хрупкие и дорогие в поддержке: полное покрытие ими даёт прогон на часы и поток ложных падений, из-за которых команда перестаёт верить результату.

Раздел 3

П2. Валидный с 2-значным регионом: A123BV77 — принимается. Запрещённая буква Ж — отклоняется. A12BV77 — неверный формат номера (две цифры вместо трёх) — отклоняется. Пустое поле: «» — отклоняется с сообщением об обязательности, если поле обязательно. Латиница A123BV77 — ловушка: визуально номер выглядит правильно, но символы другие. Решение принимает не тестировщик: нужно уточнить у аналитика, приводит ли система латиницу к кириллице (частая практика для номеров) или отклоняет. Ваша задача — заметить неоднозначность и завести вопрос.

П3. K3 — кнопка не видна (заявка в архиве). K4 — кнопка не видна (статус терминальный). Дополнительно стоит завести проверку, что при скрытой кнопке запрос отзыва, отправленный напрямую в API, тоже отклоняется: скрытие в интерфейсе не является защитой.

П4. Пример решения. 1) Из «Отозвана» в «В работе»: PATCH должен вернуть 409 (конфликт состояния) или 422; конкретный код смотрим в контракте API, главное — не 200. 2) Из «Завершено» в «Согласование»: тот же запрос, ожидаем отказ и неизменный статус в базе. 3) Из «Непрочитано» сразу в «Завершено» минуя согласование: ожидаем отказ. В каждом случае после запроса проверяем SQL-запросом, что статус в базе не изменился.

T3. 1) 9, 10, 11, 98, 99, 100. 2) Таблица решений. 3) вариант а. 4) Важнее запрещённые: разрешённые переходы обычно проверяются самим ходом работы, а дыра в запрете — это уязвимость и порча данных.

Раздел 4

П5. Пример исправленного репорта: заголовок «Ошибка 500 при открытии заявки из архива (роль Сотрудник)»; окружение — стенд, сборка, браузер, роль; предусловия — есть архивная заявка организации пользователя; шаги — 1. Войти как sergey, 2. Открыть «Архив», 3. Кликнуть по заявке № 1042; факт — страница не открывается, GET /api/applications/1042 отвечает 500, в консоли ошибка; ожидаемо — карточка заявки открывается в режиме просмотра; вложения — скриншот, HAR, лог. Severity разумно поставить «критический», а не «блокер»: сломан один раздел, остальная система работает. Блокер — когда работать нельзя вообще.

П6. Возможные пункты: доступ — форма недоступна пользователю без права подачи; поля — обязательные поля подсвечиваются, дата выезда не раньше даты въезда, прошедшая дата не выбирается, номер машины валидируется; отправка — кнопка блокируется на время запроса, при ошибке сети данные формы не теряются; негатив — заявка без сотрудников не отправляется, длинные значения обрезаются или отклоняются, спецсимволы не ломают отображение, обновление страницы после отправки не создаёт дубль.

T4. 1) Чек-лист называет, что проверить; тест-кейс описывает, как именно, с данными и ожидаемым результатом. 2) Тестировщик после ретеста. 3) б. 4) В окружение (роль пользователя) и обязательно в заголовок, если дефект специфичен для роли.

Раздел 5

П7. 1) Порт 443: его задаёт схема `https`, браузер подставляет по умолчанию. 2) Ожидается редирект 301 на `https`-адрес; если редиректа нет и страница открывается по `http` — это дефект, пароли и токены пойдут открытым текстом. 3) Жёсткая перезагрузка `Ctrl+Shift+R`, галка `Disable cache` в DevTools, приватное окно; если после сброса всё верно — дефект в настройках кэширования. 4) Страница по `HTTPS` запрашивает ресурс по `HTTP`, браузер его блокирует. Чаще всего это дефект фронта (жёстко прописанный `http`-адрес) либо конфигурации: адрес формируется на бэкенде без учёта схемы.

П8. Старая вкладка после выхода: проверить в Network, отдаёт ли API данные по старому токenu — если да, дефект инвалидации сессии, `severity` высокий. Не загрузились иконки с ошибкой блокировки: смешанный контент или CORS, смотреть точный текст в консоли. Падения только при параллельном запуске: сработал лимит частоты запросов, код 429; лечится тестовыми лимитами на стенде. Ссылка из письма работает, из приложения нет: в приложении формируется адрес с внутренним портом или без схемы — сравнить две ссылки посимвольно.

Т5. 1) `HTTPS` — тот же `HTTP` внутри зашифрованного канала TLS; `HTTP` порт 80, `HTTPS` порт 443. 2) Сервер не помнит предыдущие запросы, поэтому идентификатор пользователя передаётся с каждым запросом: `cookie` сессии или токен в заголовке. 3) в — якорь после `#`. 4) Блокер: браузер показывает предупреждение вместо продукта, пользоваться системой нельзя.

Раздел 6

П9. Двойное нажатие — фронтенд, ловится UI-тестом (и дублем в базе через SQL). Доступ чужой организации через API — слой прав в API, ловится API-тестом, интерфейс тут вообще ни при чём. Сохранение без обязательного поля — валидация на бэкенде, дешевле всего юнит- или API-тестом. Уведомление через 20 минут — очередь и фоновая обработка, ловится проверкой времени выполнения задачи, а не UI-тестом.

Т6. 1) Тяжёлые операции нельзя делать в ответе на запрос: пользователь ждёт, таймауты рвут соединение; очередь позволяет ответить сразу и обработать в фоне. 2) Интерфейс может просто прятать кнопку, а сам запрос при этом выполняется: проверять надо на уровне API. 3) Запрос принят, но обработка асинхронна и результат этим ответом не гарантирован.

Раздел 7

П10. Возможные критерии: кнопка не видна, если заявка в архиве или в терминальном статусе; после подтверждения диалога заявка меняет статус на «Отозвана» и исчезает из активных; повторный отзыв невозможен, кнопка пропадает; запрос отзыва от пользователя, не являющегося автором, возвращает 403 или 404; действие фиксируется в журнале аудита.

П11. Порядок: 2, 3 (`git status`), 5 (`git add .`), 4, 1. То есть: создать ветку, посмотреть изменения, добавить файлы, зафиксировать коммит, отправить ветку.

Т7. 1) DoR — условия, при которых задачу можно взять в работу; DoD — при которых её можно считать законченной. 2) На проде: там живые пользователи и настоящие данные, эксперимент означает реальный ущерб. 3) Чтобы включать и выключать функцию без нового релиза и быстро гасить проблему. 4) Например: на стенде другие данные и объёмы, а на бое всплыл случай, которого в тестовых данных не было; либо сломался сам процесс выката, и до пользователей доехала старая или неполная сборка.

Раздел 8

П12. 1) Вошёл в систему, создал заявку (успешно, 201), затем открыл заявку 1042 и получил ошибку. 2) Дефект — строка `ERROR` с кодом 500; `WARN` про истечение срока вложения это предупреждение бизнес-логики, не ошибка. 3) Например: «500 при открытии карточки заявки 1042 (`nil pointer в buildApplicationResponse`)». 4) `request_id=e5f6` — по нему разработчик найдёт весь путь запроса.

Т8. 1) `WARN` — «странно, но работа продолжается»; `ERROR` — операция не выполнена. 2) Он связывает клик пользователя, конкретный `HTTP`-запрос и записи в логах разных сервисов. 3) Смотрите не то время (логи в UTC) или не тот сервис либо окружение; ещё вариант — уровень логирования на стенде поднят и `INFO` не пишется.

Раздел 9

T9. 1) Событие — любая запись из системы; инцидент — событие или цепочка, означающая возможную атаку и требующая реакции. 2) Собирает из потока событий осмысленную последовательность и создаёт инцидент. 3) Интерфейс лишь прячет элементы, а запрос к API можно отправить напрямую: проверка прав должна работать на сервере. 4) Да: для продуктов кибербезопасности журнал аудита — часть функциональности, а не служебная мелочь.

P22. Проверяйте себя по таблице «Критерии готовности» из раздела 16: ни одного селектора в тестах, ни одного `waitForTimeout`, каждый тест проходит в одиночку, есть позитивный и негативный сценарий, есть API-тест, история коммитов показывает ход работы, README запускается на чужой машине.

Раздел 10

P13. Карточка — `[data-testid="application-card"]`; статус — `[data-testid="application-status"]`; кнопка — по роли и имени: в Playwright `getByRole('button', { name: 'Отозвать' })`, в CSS пришлось бы использовать класс, что хуже; заголовок — по тексту: `getByText('Заявка № 1042')`, либо XPath `//h3[contains(text(), 'Заявка №')]`.

P14. `this.dateFrom = page.getByTestId('application-date-from');`; `this.dateTo = page.getByTestId('application-date-to');`; `this.submitButton = page.getByTestId('application-submit');`; .Метод:

```
async fillAndSubmit({ organization, from, to }) {
  await this.organizationSelect.selectOption(organization);
  await this.dateFrom.fill(from);
  await this.dateTo.fill(to);
  await this.submitButton.click();
}
```

T10. 1) Network: ушёл ли запрос вообще и с каким кодом ответа; если запроса нет — дефект на фронте. 2) Класс отвечает за оформление и меняется при редизайне; кроме того, таких кнопок на странице может быть несколько. 3) в — ассерты внутри каждого метода РОМ делают его негибким; допустимы отдельные expect-методы, но не проверка внутри действия. 4) Локаторы завязаны на вёрстку (классы, вложенность), а не на устойчивые признаки.

Раздел 11

P15. 1) 201 Created. 2) Начать с API: если создание вернуло 201 и объект, проверить запрос списка — возможно, фильтр по организации или пагинация отсекают новую заявку; затем сверить строку в базе. 3) `expect(body.data.status).toBe('Непрочитано');`

P16. `toBe(201)`, `toBe(true)`, `toBe('Непрочитано')`. Второй тест:

```
test('без токена создание запрещено', async ({ request }) => {
  const res = await request.post('/api/applications', {
    data: { organization_id: 7 },
  });
  expect(res.status()).toBe(401);
});
```

T11. 1) Пользователь аутентифицирован, но не имеет прав на объект или действие. 2) API-тест: он проверяет то же правило без браузера, за доли секунды и без зависимости от вёрстки. 3) в — сверяться с контрактом; если контракт требует 4xx, это дефект. 4) Чтобы разработчик воспроизвёл запрос у себя одной командой, без пересказа шагов.

Раздел 12

P17. а) `SELECT status, COUNT(*) AS cnt FROM applications WHERE organization_id = 7 GROUP BY status;` б) `LEFT JOIN` и `WHERE a.id IS NULL`; в) `WHERE status = 'В работе' AND accepted_at IS NULL`;

T12. 1) WHERE фильтрует строки до группировки, HAVING — уже посчитанные группы. 2) б. 3) Открыть Network и посмотреть код ответа на создание: при 2xx дефект на бэкенде (не сохранил), при 4xx/5xx фронт показал успех вместо ошибки — это тоже дефект, и severity высокий, потому что пользователь уверен, что заявка подана.

Раздел 13

П18. 1) У функции теста нет `async`, хотя внутри есть `await`. 2) `form.goto()` без `await`. 3) `const count: string = 1;` — число присвоено переменной типа `string`. Плюс полезная придирка: `expect(...)` с локатором тоже требует `await`.

П19. `success: boolean; data: T; error?: string;` и `function unwrap<T>(body: ApiResponse<T>): T { return body.data; }`

Раздел 14

П20. 1) `.toBeHidden()`. 2) `.toBeVisible()` или `.toHaveText('Заявка отправлена')`. 3) `.toHaveText('Непрочитано')`. 4) `.toHaveURL('/applications/')`.

П21. Причины: `waitForTimeout(1500)` — гадание по времени; `networkidle` — не наступает на странице с фоновыми запросами; локатор по вложенности `div.filters > div:nth-child(2)` ломается от любой правки вёрстки; жёсткое число 7 зависит от данных, оставшихся от других тестов. Вариант решения:

```
test('фильтр по статусу', async ({ page }) => {
  await page.goto('/applications');
  await page.getByTestId('filter-status-unread').click();
  const rows = page.getByTestId('application-card');
  await expect(rows.first()).toBeVisible();
  await expect(rows.filter({ hasText: 'Непрочитано' }))
    .toHaveCount(await rows.count());
});
```

Смысл последней проверки: не «строк ровно семь», а «все показанные строки соответствуют фильтру».

T13. 1) Все действия и проверки уже содержат ожидание нужного состояния; пауза лишь замедляет прогон и не гарантирует готовности. 2) Трейс показывает состояние страницы на каждом шаге, сетевые запросы и консоль, а не один кадр в момент падения. 3) б. 4) Фикстура убирает копипасту, гарантирует одинаковое стартовое состояние и умеет прибирать данные после теста. 5) Через интерфейс — проверяется и сам путь создания, данные точно проходят валидацию; через API — быстрее и стабильнее, тест не падает из-за чужой формы. Обычно: то, что проверяем, — через интерфейс, фон — через API.

Раздел 15

T14. 1) Дешёвые и быстрые проверки отсекают большинство дефектов раньше, чем запускается дорогой E2E-прогон. 2) Разобрать: найти причину, починить тест или завести дефект продукта; перезапуск маскирует проблему. 3) Флак: красный тест чинят, а флак приучает игнорировать красный цвет, и вместе с ним пропускают настоящие падения.

Б Шпаргалки

Коды ответов

200 успех	201 создано	204 успех без тела	400 плохой запрос
401 не вошёл	403 нет прав	404 не найдено	409 конфликт
422 не прошло валидацию	429 слишком часто	500 ошибка сервера	502/503 сервис недоступен

Playwright: то, что нужно каждый день

```
page.goto('/applications')
page.getByRole('button', { name: 'Отправить' })
page.getByTestId('application-card')
page.getByText('Непрочитано')
locator.fill('текст') locator.click() locator.selectOption('7')
await expect(locator).toBeVisible() / toHaveText() / toHaveCount(n)
await expect(page).toHaveURL(/regex/)
await expect.soft(locator).toHaveText('...')
await expect(async () => { ... }).toPass()
npx playwright test --headed --debug
npx playwright show-report / show-trace trace.zip
```

SQL для проверок

```
SELECT ... FROM t WHERE ... ORDER BY ... LIMIT n;
JOIN / LEFT JOIN ... ON ...
GROUP BY ... HAVING COUNT(*) > n
WHERE col IS NULL -- сравнение с NULL только так
WHERE ts > now() - interval '1 minute'
```

Шаблон баг-репорта

Заголовок: [что сломано] при [условие], [где]
Окружение: стенд, сборка, браузер, роль
Предусловия: ...
Шаги: 1. ... 2. ... 3. ...
Факт: ... (код ответа, текст ошибки)
Ожидаемо: ... (ссылка на требование)
Вложения: скриншот / HAR / трейс / лог консоли
Severity: блокер | критический | значительный | незначительный | тривиальный

Техники тест-дизайна: что когда

поле с диапазоном	→ классы эквивалентности + границы
несколько условий	→ таблица решений
объект меняет статусы	→ диаграмма состояний и переходов
много настроек	→ pairwise
нет требований	→ исследовательское + предугадывание ошибок

Порядок разбора «что-то не работает»

1. Что видно на экране? (текст ошибки, состояние формы)
2. Network: ушёл ли запрос, метод, код ответа, тело
3. Console: есть ли ошибка JavaScript
4. База: появилась ли строка, какие значения
5. Вывод: слой дефекта → баг-репорт с точным адресом

Мой словарь

Выписывайте сюда термины, которые встретились в работе и в вакансиях: своими словами, а не определением из интернета. Проговорённый своими словами термин на собеседовании звучит иначе, чем заученный.

Термин	Что это значит своими словами

Журнал прогонов

Заполняется на этапе рет-проекта: по нему видно, растёт ли набор и стабильны ли тесты.

Дата	Тестов	Упало	Что чинил

ИТОГ РАЗДЕЛА

Тетрадь закончилась, набор навыков — нет. Дальше растут через задачи: чем больше форм вы разложите на классы и границы, тем быстрее это становится автоматическим. Успехов на стажировке.